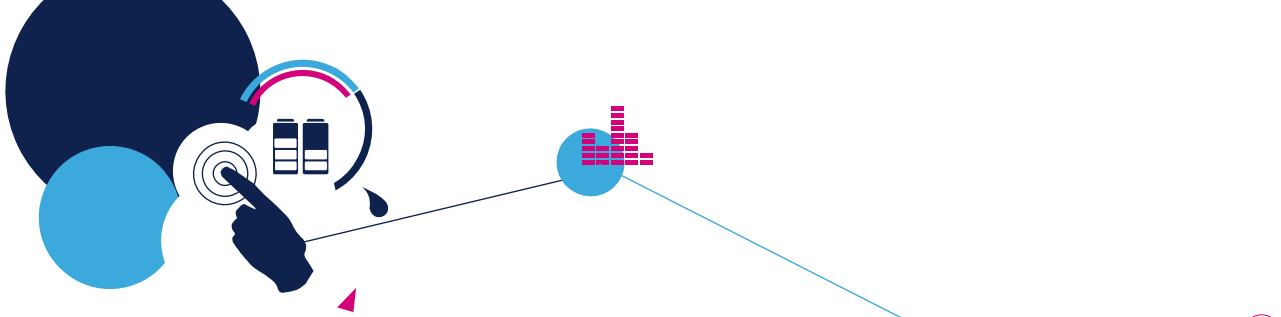


STM32L4 Tips & Tricks

STM32L4 workshop



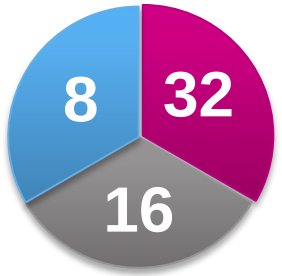
life.augmented

AudioPlayer optimization

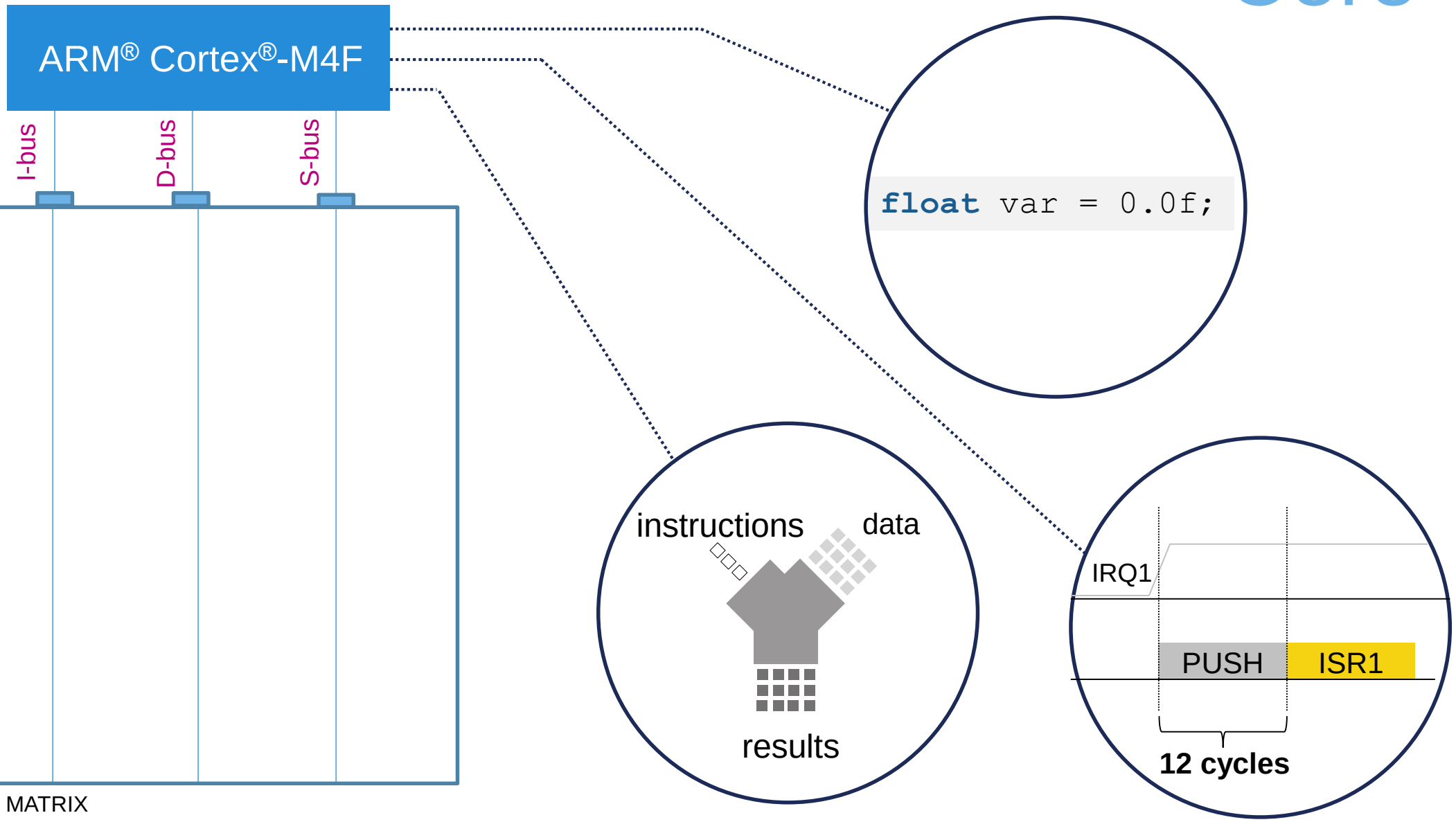
- Let’s continue our development of AudioPlayer and let’s make it more **green**.
- In our next steps we will work on **AudioPlayer.ioc** CubeMX project after STEP6 and its generated project.
- During each step we will measure the average current consumption during PLAY and STOP phases of our application.
- Please connect your meter to JP5 (pins 1-2) and STM32L4-Discovery board via USB-mini to PC (STLink side).

Squeeze the maximum

1. **From the architecture**



life.augmented


ARM® Cortex®-M4F
I-bus
D-bus
S-bus
BUS MATRIX
float var = 0.0f;
instructions
data
results
IRQ1
PUSH
ISR1
12 cycles

Overview



Typ @ VDD =1.8 V @ 25 °C

** : with RTC*

*** : from SRAM1*

FlexPowerControl

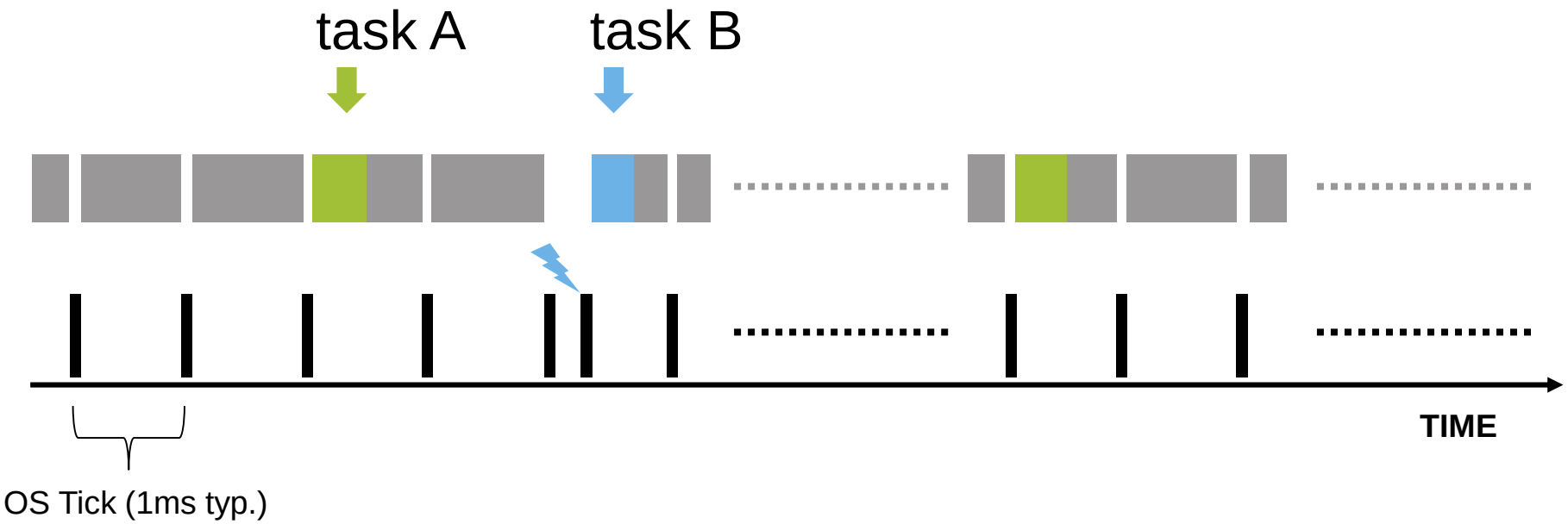
- Efficient running
- 7 low-power modes, several sub-modes
- High flexibility

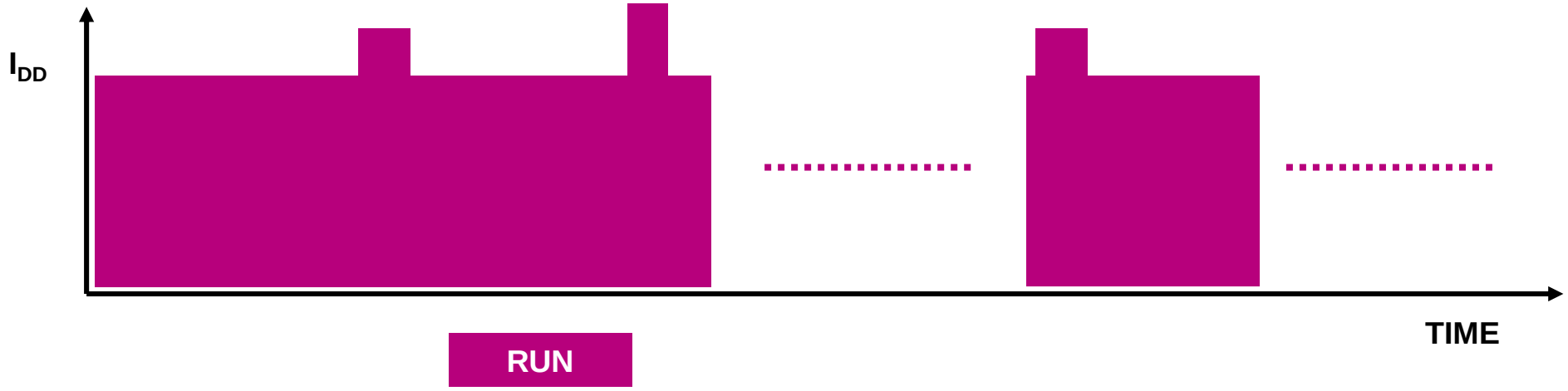
Application benefits

- High performance
  - ➔ CoreMark score = 273
- Outstanding power efficiency
  - ➔ ULPBbench score = 150

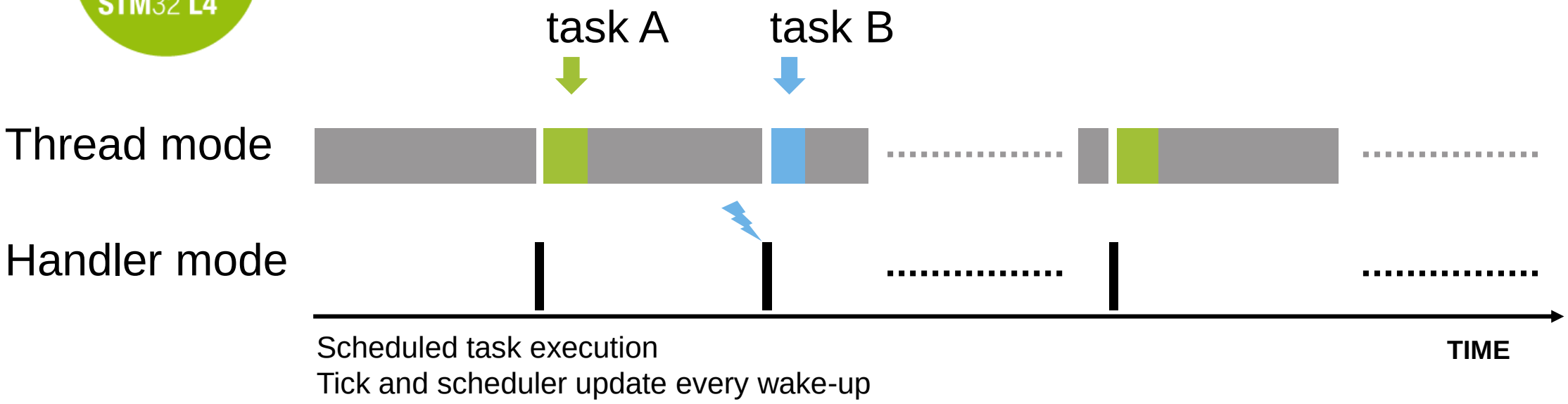
life.augmented

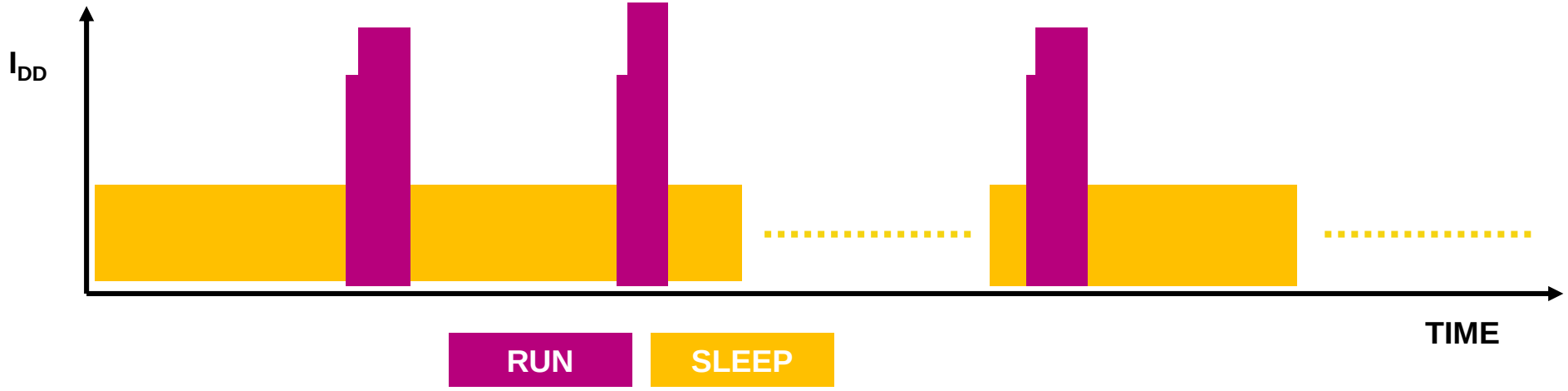
RTOS normal mode





RTOS Tickless mode



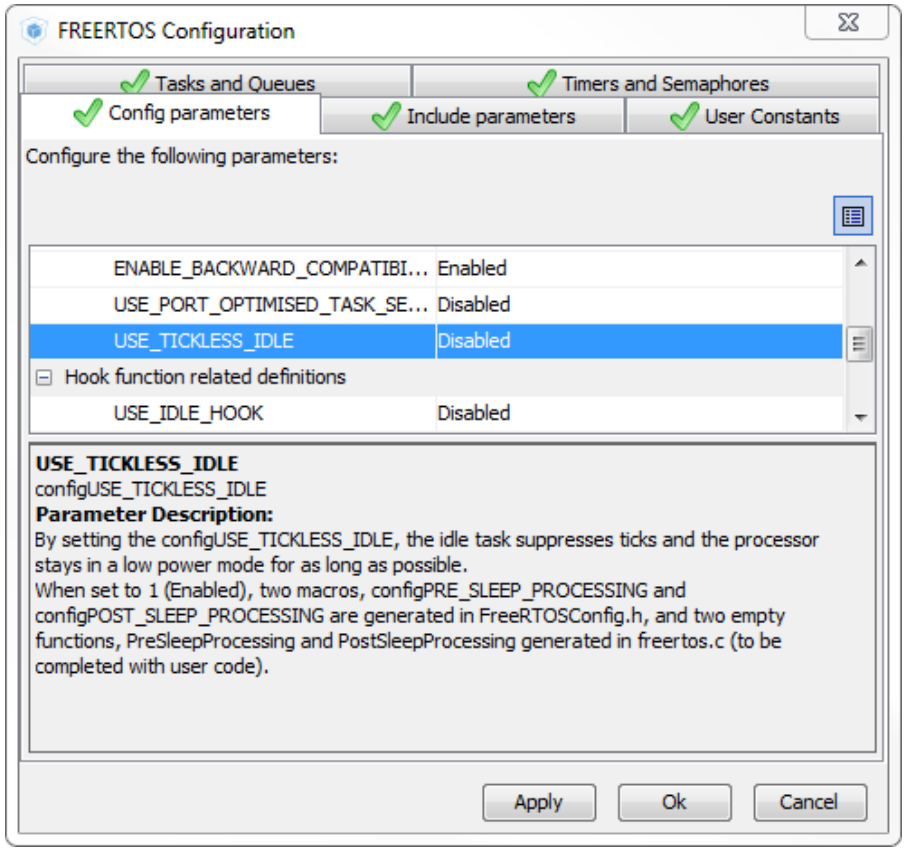


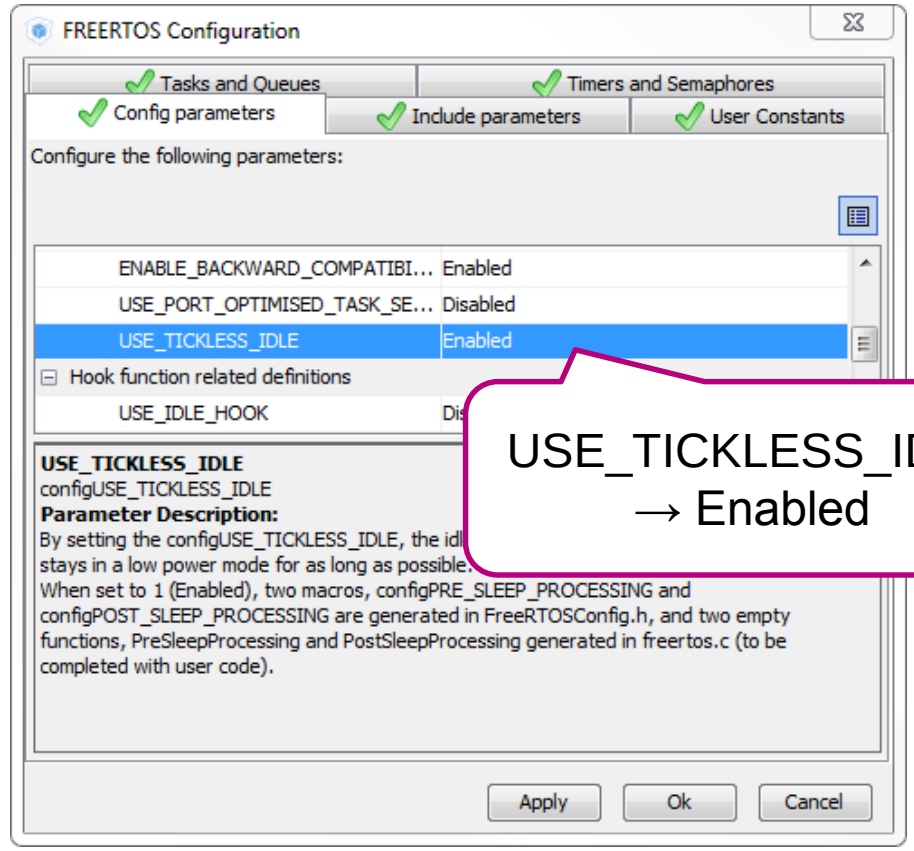
Tickless Mode in FreeRTOS

- Kernel can stop system tick interrupt and place MCU in low power mode, on exit from this mode systick counter is updated
- Enabled when setting configUSE_TICKLESS_IDLE as 1
- The kernel will call a macro portSUPPRESS_TICKS_AND_SLEEP() when the Idle task is the only task able to run (and no other task is scheduled to exit from blocked state after n ticks)
  - n value is defined in FreeRTOSconf.h file
- FreeRTOS implementation of portSUPRESS_TICKS_AND_SLEEP for cortexM3/M4 enters MCU in sleep low power mode
- Wakeup from sleep mode can be from a system interrupt/event

life.augmented

How to enable Tickless mode?





Apply small patch

- Apply patch from folder
- **C:\STM32L4_Workshop\2017_HandsOn\HandsOn\3_Tips_and_Tricks\Patch** project (C:\AudioPlayer\AudioPlayer\)
- It contains the following files:

  .\Src\

  usbd_conf.c
  user_diskio.c
  sai.c

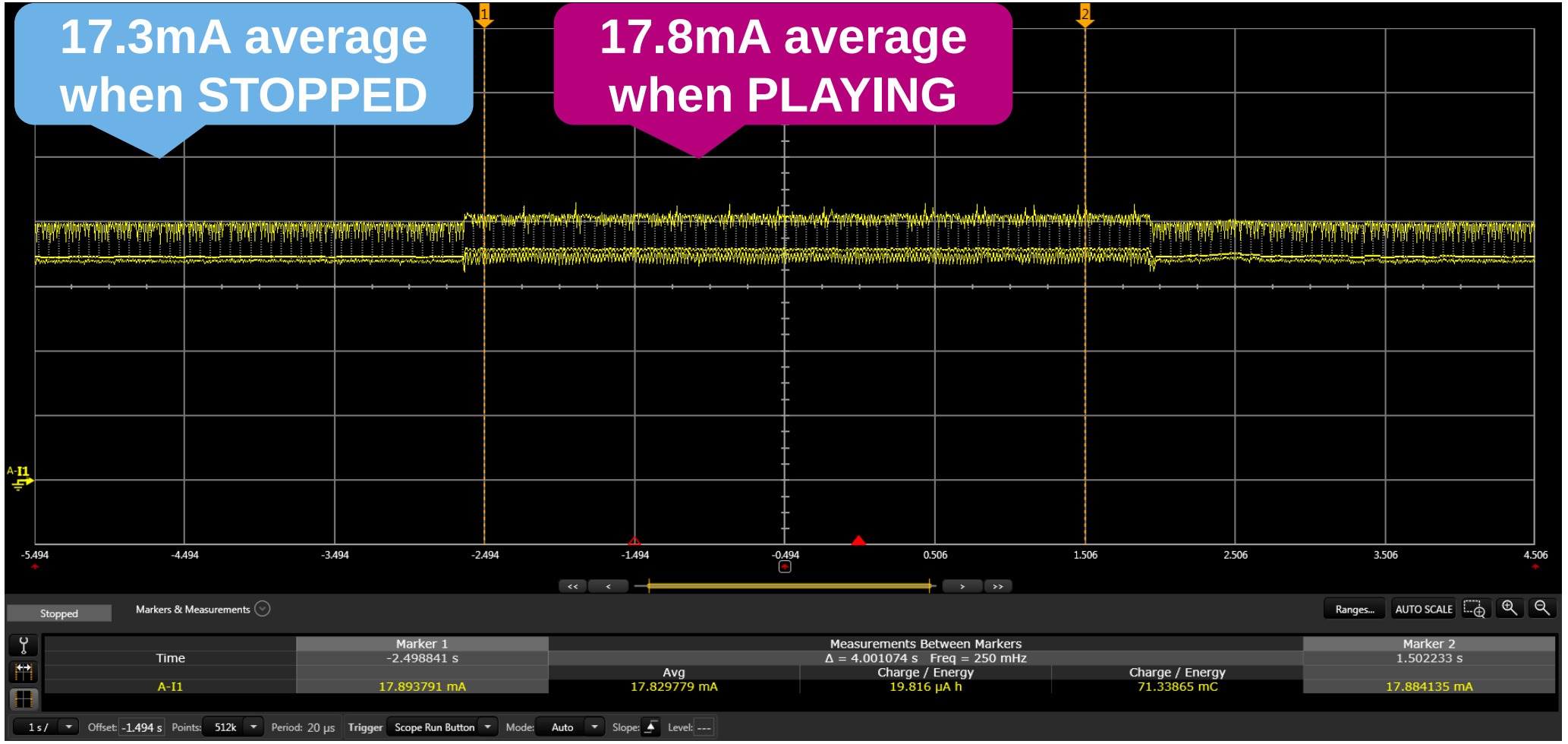
This operation is STM32CubeMX non-intrusive. You can re-generate the project later and the modifications will be retained. (except sai.c and user_diskio.c)

1 Let’s try it out

1. Open AudioPlayer.ioc CubeMX project after last (6) step
2. Enable tickles mode within FreeRTOS configuration (like below)
3. Generate the project
4. Apply Patch.
5. Within **PreSleepProcessing()** function in **freertos.c** file call the function which would put the MCU into low power mode (SLEEP).
6. Compile and run the code
7. Measure the current consumption in PLAY and STOP player modes

AudioPlayer consumption

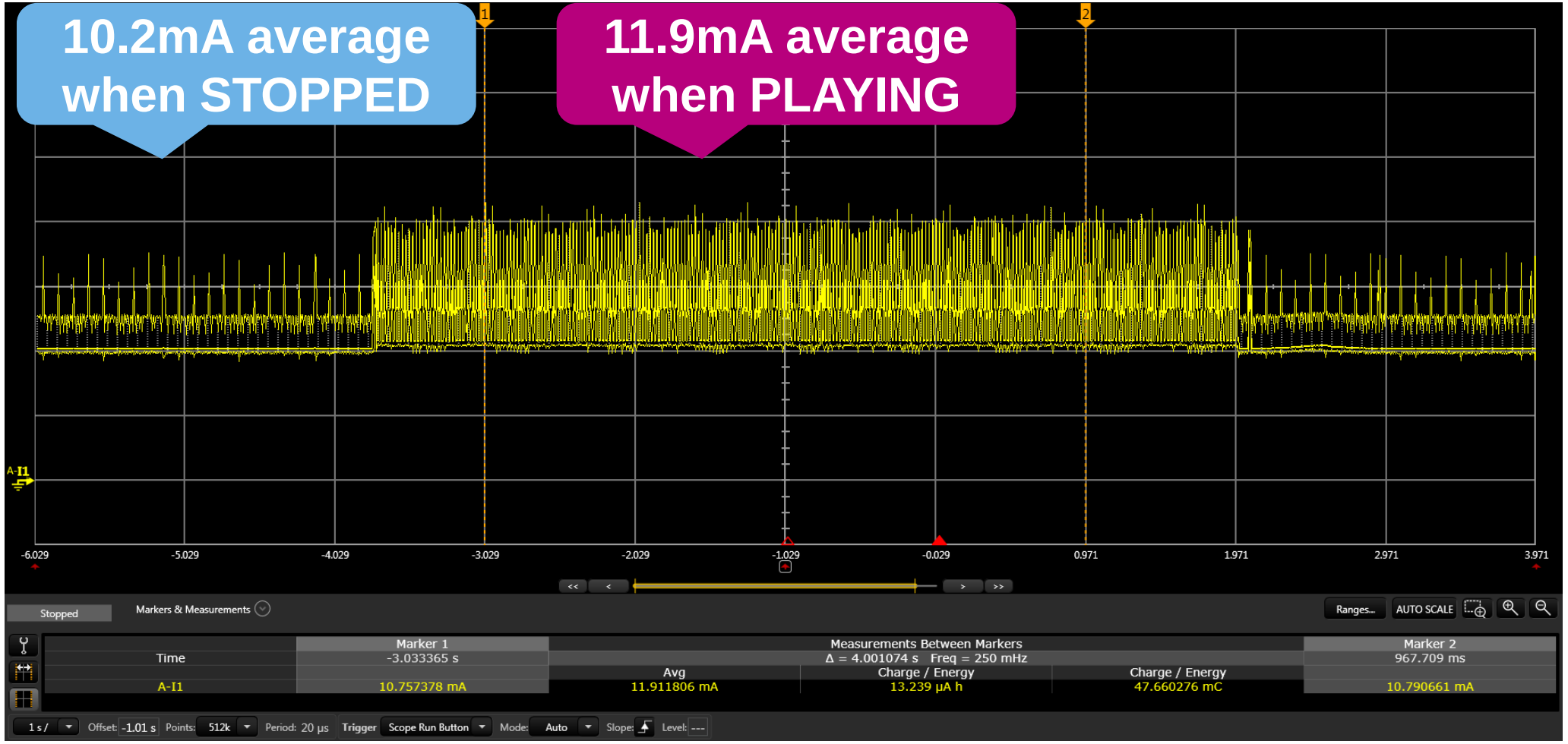
Original state (STEP6)



life.augmented

AudioPlayer optimization 1/5

RTOS Tickless Mode used



life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	….. mA	….. %	…… mA	…… %

Just the MCU consumption considered
(including circuitries supplied from GPIOs)

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

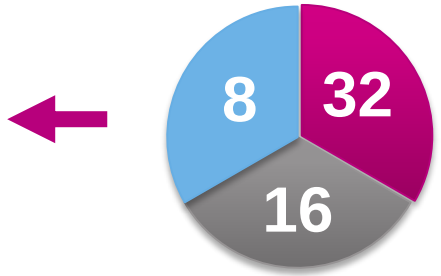
Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%

Just the MCU consumption considered
(including circuitries supplied from GPIOs)

life.augmented

Squeeze the maximum

1. **From the architecture**



2. **From the peripherals**



Peripheral Clock Gating

- The clock tree toward each register increases consumption, when the Bus clock is running…
- So clock toward each peripheral register group can be gated (Default is gated)

IP	range 1	range 2	
GPIOA	4.8	3.8	
GPIOB	4.8	4.0	
CRC	0.4	0.2	
DMA1	1.4	1.3	
FMC	8.9	7.5	
SYSCFG	0.6	0.4	
TIM1	8.3	6.9	
TIM17	3.0	2.4	µA/MHz
SAI1	4.7	3.9	
QUADSPI	7.8	6.7	
LCD	1.0	0.8	
WWDG	3	2.5	
USB	39.6	N/A	
PWR	0.5	0.5	
DAC1	2.4	1.9	
…….			
ALL	256.8	189.6	

More information can be found in datasheet (Table 39: Peripheral current consumption)

IP Behavior when clock is gated 1/2

IP	Clock gated	Available interrupt
LCD	Display is running	
OpAmp	Output ok	
ADC	Conversion on going but data not stored	
DAC	Output ok, but value stays still	
COMP	Output ok	COMP1, COMP2
GPIO	Output stay still, analog input ok	EXTIs
RTC	running	Tamper/Timestamp Wake-Up/Alarm
IWDG	running	Reset
SYSCFG	Stays still, except pending request reg	PVD

life.augmented

IP Behavior when clock is gated 2/2

IP	Clock gated	Available interrupt
WWDG	stopped	
Comm	stopped	USB_FS_WKUP
TIMER	stopped	
AES	stopped	
CRC	stopped	
DMA	stopped	

Clocks gating and AudioPlayer

Enabled peripherals analysis

```
AHB1ENR      = 0x00000102
  TSCEN      = 0
  CRCEN      = 0
  FLASHEN    = 1
  DMA2EN     = 1
  DMA1EN     = 0

AHB2ENR      = 0x0000101F
  RNGEN      = 0
  AESEN      = 0
  ADCEN      = 0
  OTGFSEN    = 1
  GPIOHEN    = 0
  GPIOGEN    = 0
  GPIOFEN    = 0
  GPIOEEN    = 1
  GPIODEN    = 1
  GPIOCEN    = 1
  GPIOBEN    = 1
  GPIOAEN    = 1

AHB3ENR      = 0x00000100
  QSPIEN     = 1
  FMCEN      = 0
```

```
APB1ENR1     = 0x10200200
  LPTIM1EN   = 0
  OPAMPEN    = 0
  DAC1EN     = 0
  PWREN      = 1
  CAN1EN     = 0
  I2C3EN     = 0
  I2C2EN     = 0
  I2C1EN     = 1
  UART5EN    = 0
  UART4EN    = 0
  USART3EN   = 0
  USART2EN   = 0
  SP3EN      = 0
  SPI2EN     = 0
  WWDGEN     = 0
  LCDEN      = 1
  TIM7EN     = 0
  TIM6EN     = 0
  TIM5EN     = 0
  TIM4EN     = 0
  TIM3EN     = 0
  TIM2EN     = 0
```

```
APB1ENR2     = 0x00000000
  LPTIM2EN   = 0
  SWPMI1EN   = 0
  LPUART1EN  = 0

APB2ENR      = 0x00200001
  DFSDMEN    = 0
  SAI2EN     = 0
  SAI1EN     = 1
  TIM17EN    = 0
  TIM16EN    = 0
  TIM15EN    = 0
  USART1EN   = 0
  TIM8EN     = 0
  SPI1EN     = 0
  TIM1EN     = 0
  SDMMCEN    = 0
  FIREWALLEN = 0
```

Clocks gating and AudioPlayer

Automatic clocks gating in SLEEP mode

- Using the same code (from tickless mode) please do some modifications within the sources (no CubeMX modifications).
- Let's just make the GPIO ports, I2C1 and PWR to be automatically disabled in SLEEP mode
  - we don't need access to these peripherals in SLEEP mode
- Following code sequence to be added after configuration of all the peripherals:

```
__GPIOA_CLK_SLEEP_DISABLE();
__GPIOB_CLK_SLEEP_DISABLE();
__GPIOC_CLK_SLEEP_DISABLE();
__GPIOD_CLK_SLEEP_DISABLE();
__GPIOE_CLK_SLEEP_DISABLE();
__I2C1_CLK_SLEEP_DISABLE();
__PWR_CLK_SLEEP_DISABLE();
```

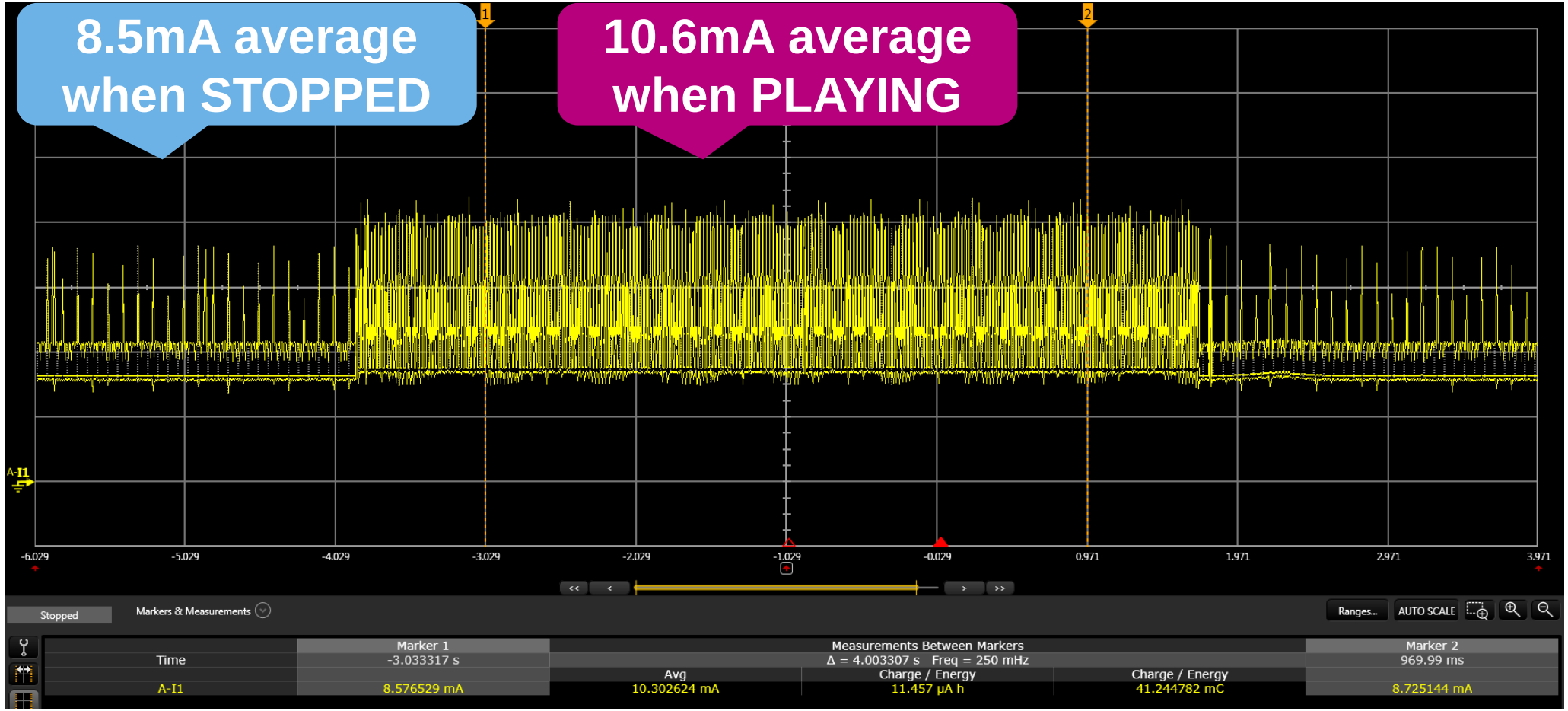
life.augmented

2 Let’s try it out

1. Open the AudioPlayer IAR project (from previous step)
2. Add the modification described on previous slide
3. Compile the code and download it to the board
4. Reset the board and measure the current consumption in PLAY and STOP modes of the player.

AudioPlayer optimization 2/5

Clocks gating applied



life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	….. mA	….. %	….. mA	….. %

Just the MCU consumption considered
(including circuitries supplied from GPIOs)

life.augmented

AudioPlayer optimization summary

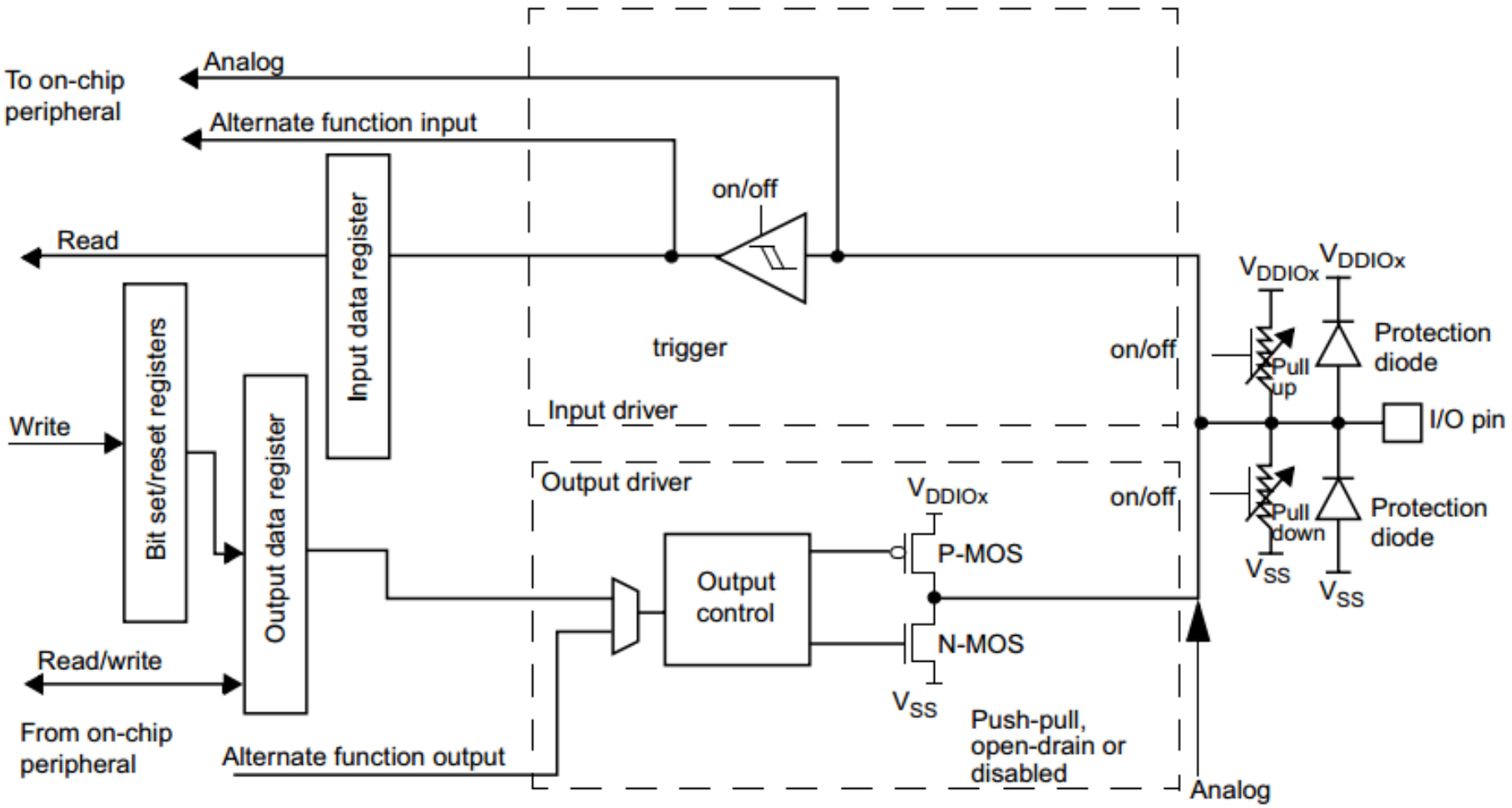
V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%

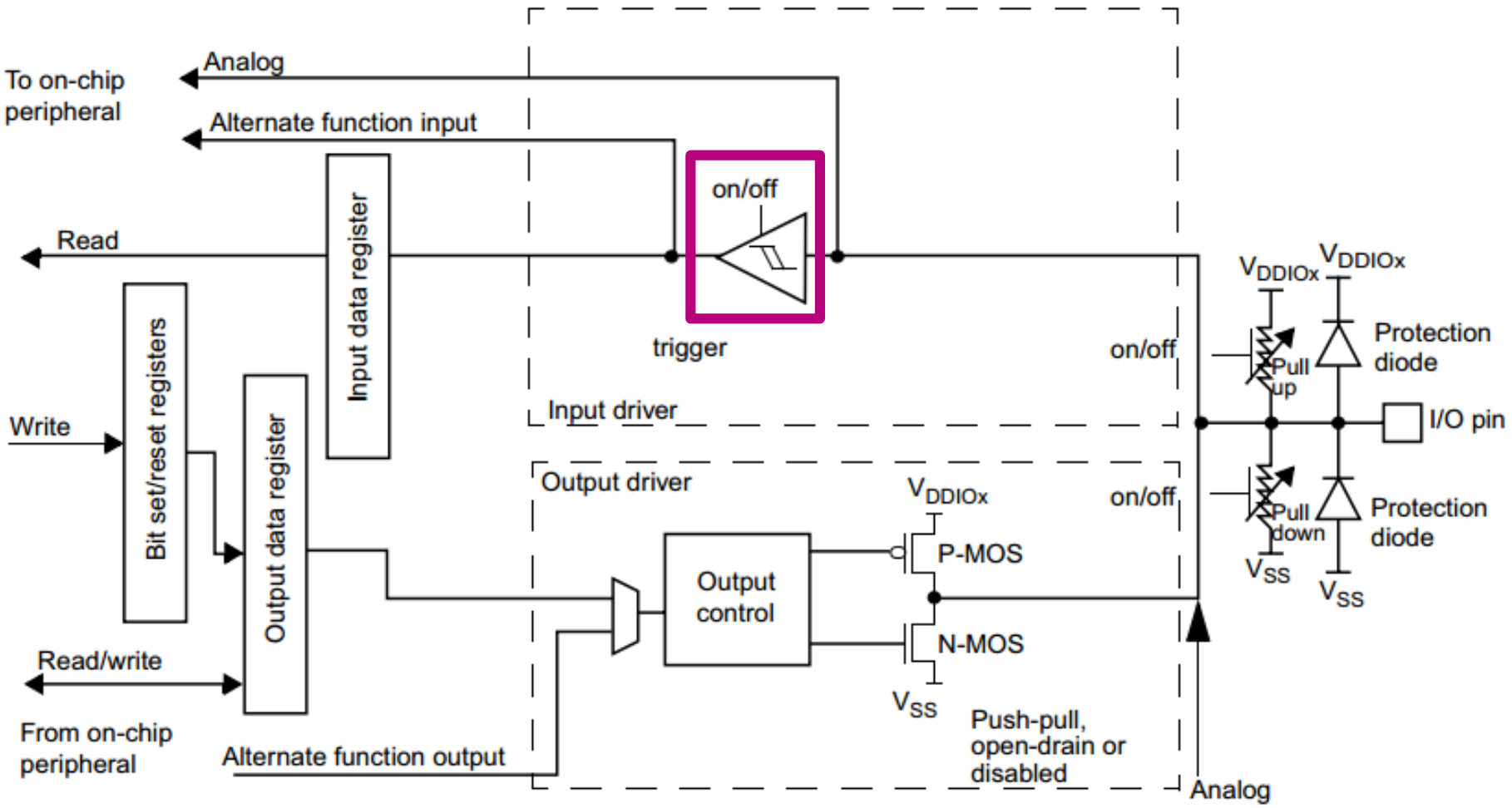
Just the MCU consumption considered
(including circuitries supplied from GPIOs)

life.augmented

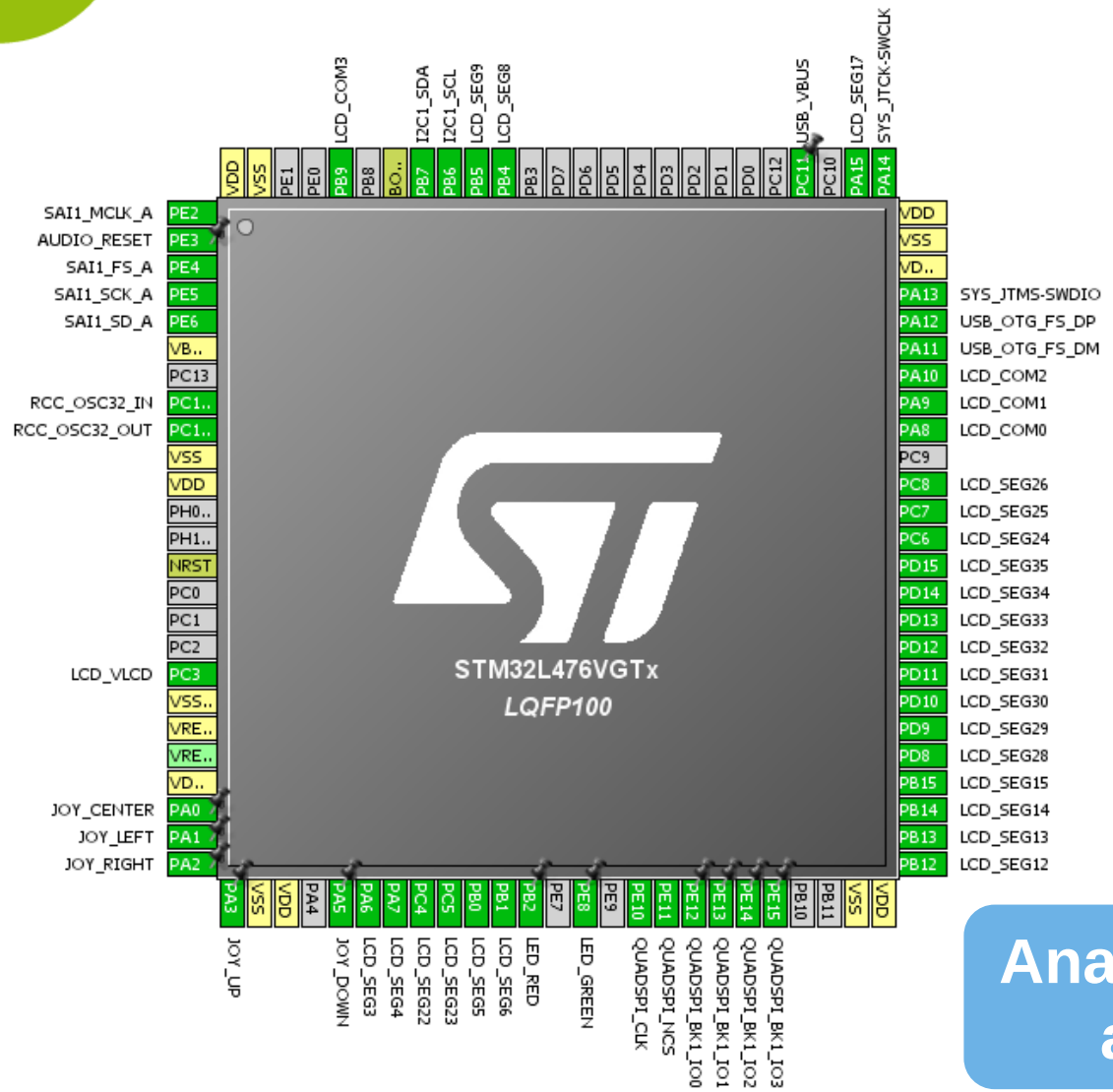
GPIOs configuration



GPIOs configuration



Unused GPIOs



9.4.1 GPIO port mode register (GPIOx_MODER) (x =A..H)

Address offset:0x00

Reset values:

- 0xABFF FFFF for port A
- 0xFFFF FEBF for port B
- 0xFFFF FFFF for ports C..G,
- 0x0000 000F for port H

31	30
MODE15[1:0]	
rw	rw

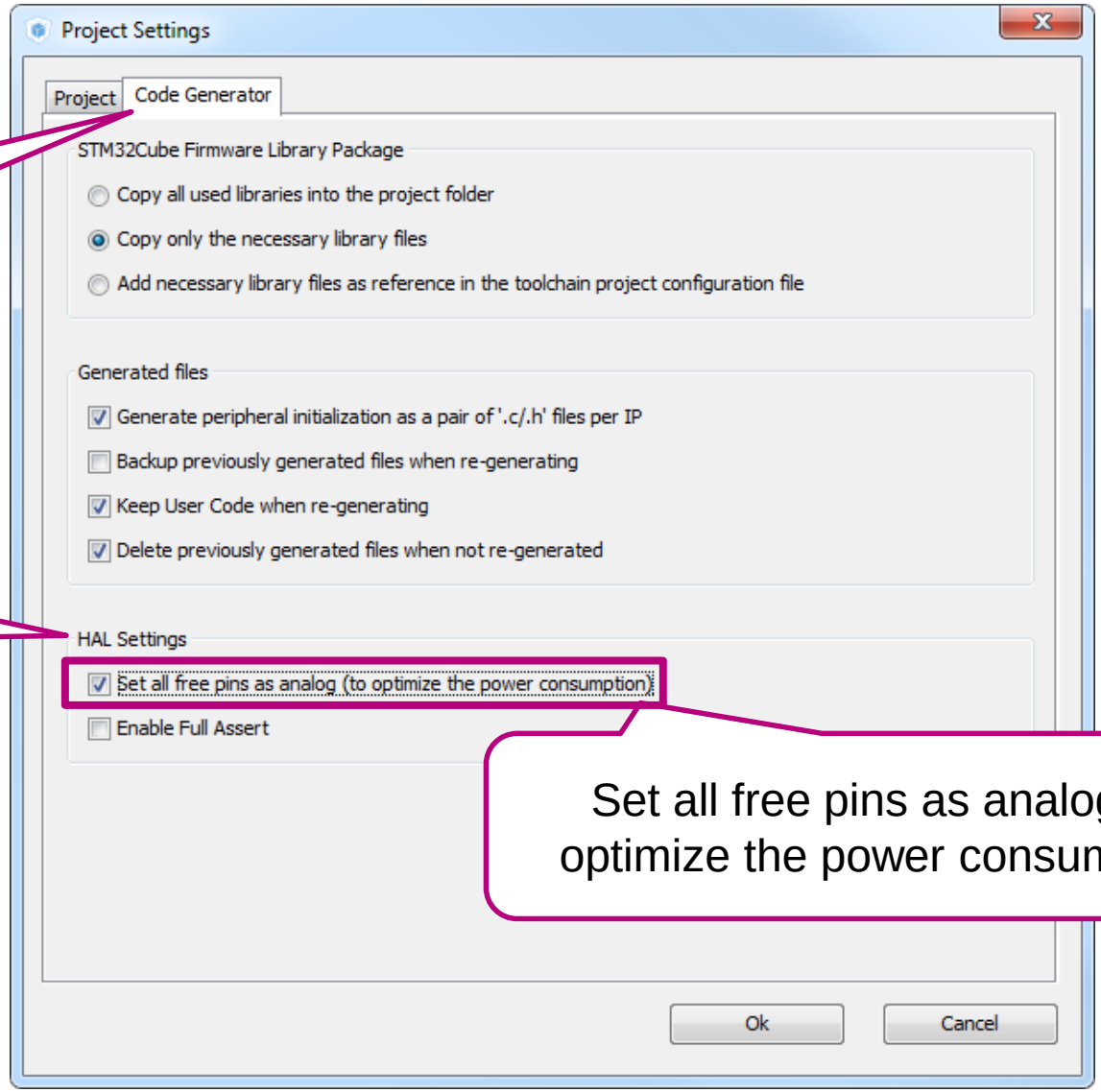
3	2	1	0
MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw

Bits 2y+1:2y **MODEy[1:0]:** Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Analog mode is default configuration after reset for most of the pins

Feature of STM32CubeMX

STM32CubeMX
→ Project Settings
→ Code Generator

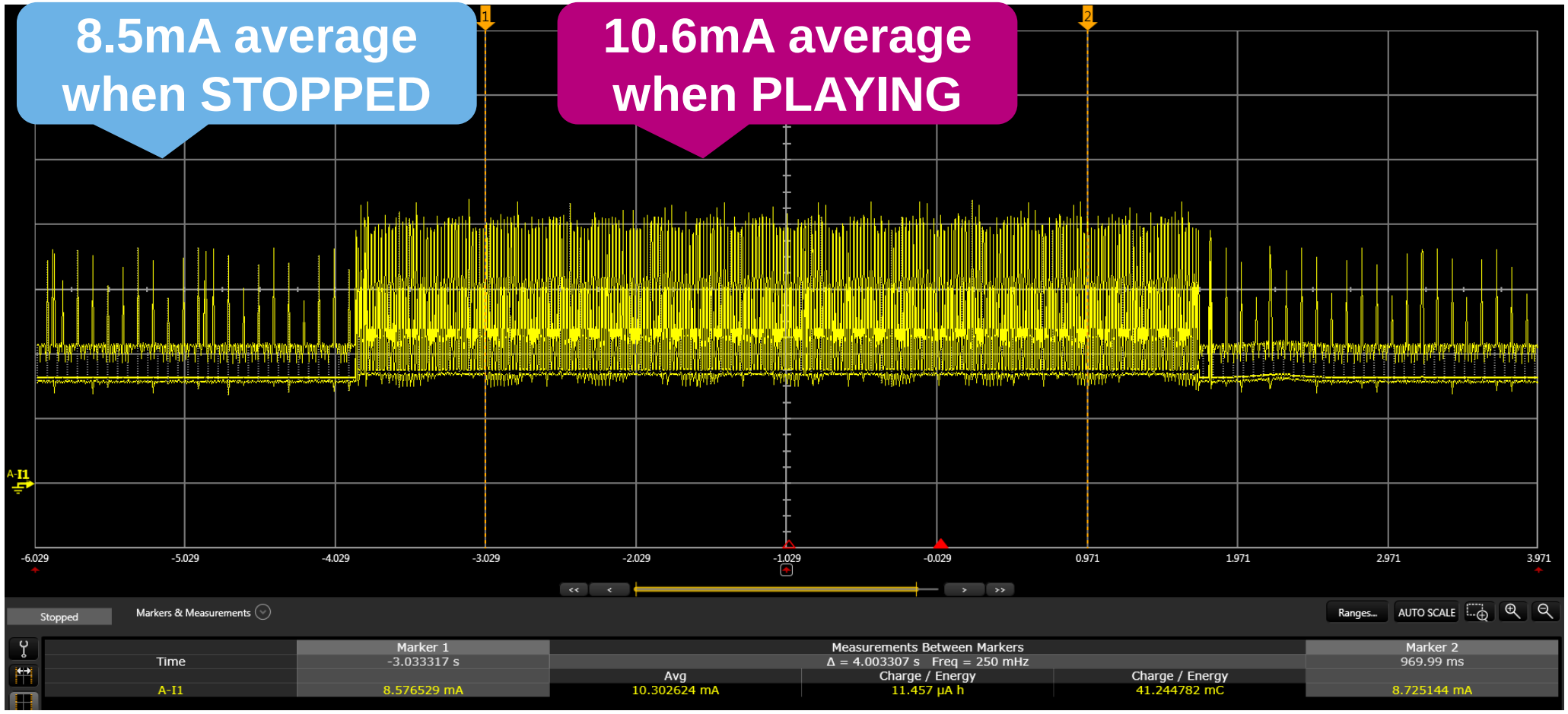


HAL Settings

Set all free pins as analog (to optimize the power consumption

AudioPlayer optimization 3/5

Unused GPIOs handled



life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	….. mA	….. %	….. mA	….. %

Just the MCU consumption considered
(including circuitries supplied from GPIOs)

life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0

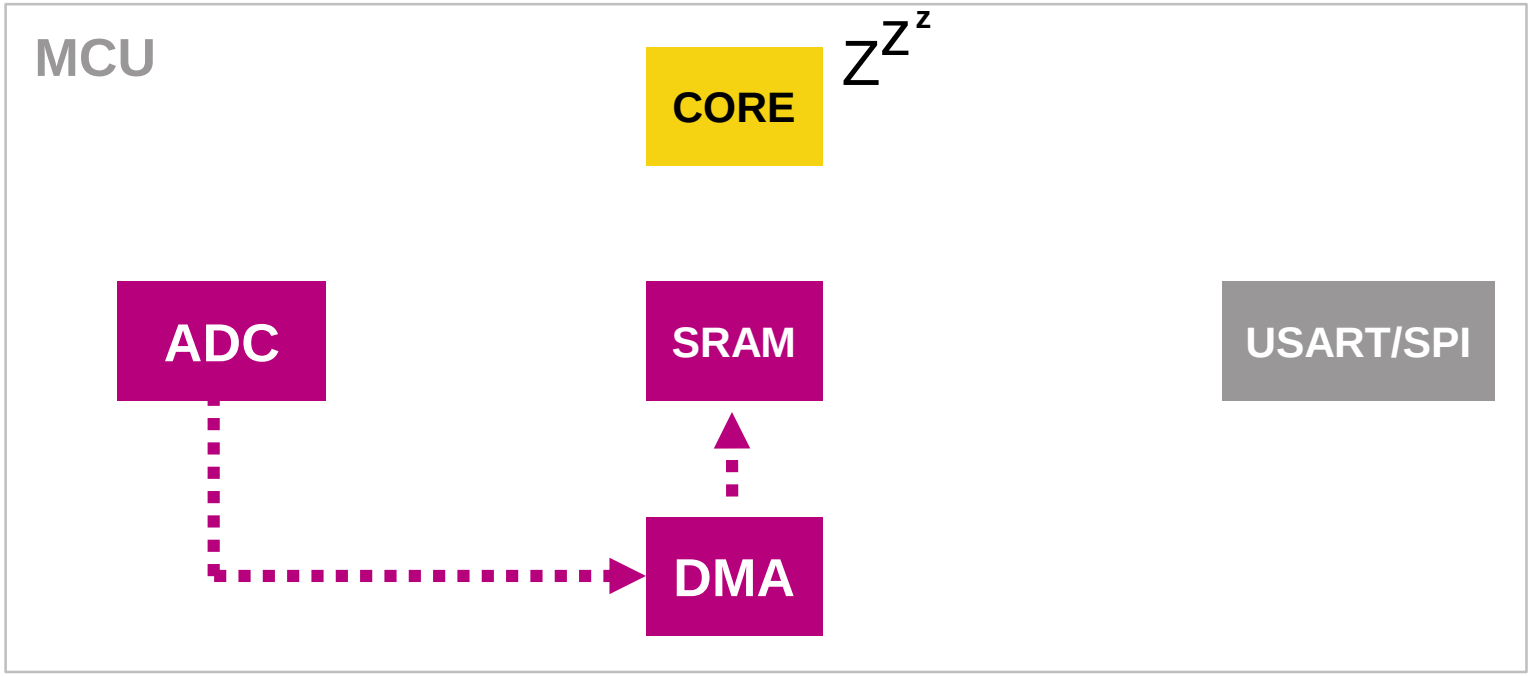
Just the MCU consumption considered
(including circuitries supplied from GPIOs)

life.augmented

Let the peripherals do the job

Scenario #1

1
SAMPLING



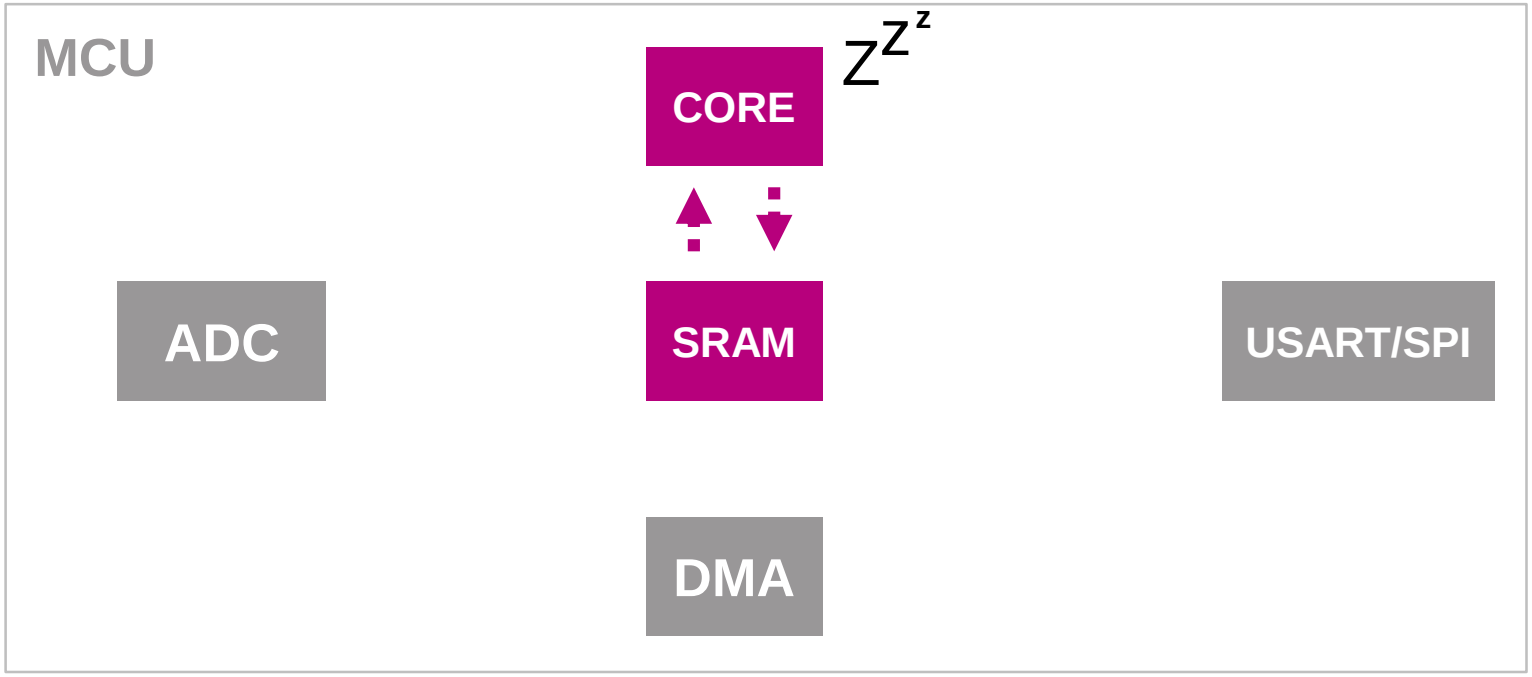


Let the peripherals do the job

Scenario #1

1
SAMPLING

2
PROCESSING





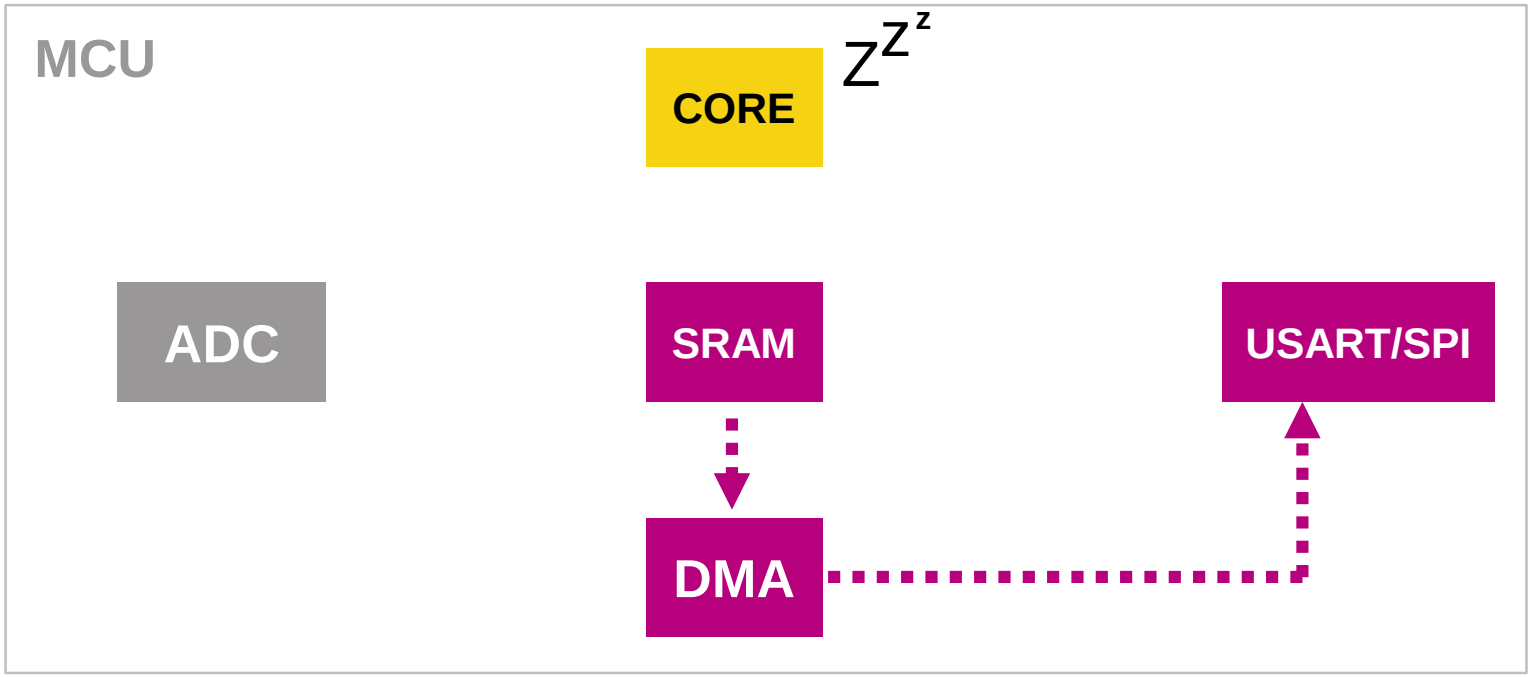
Let the peripherals do the job

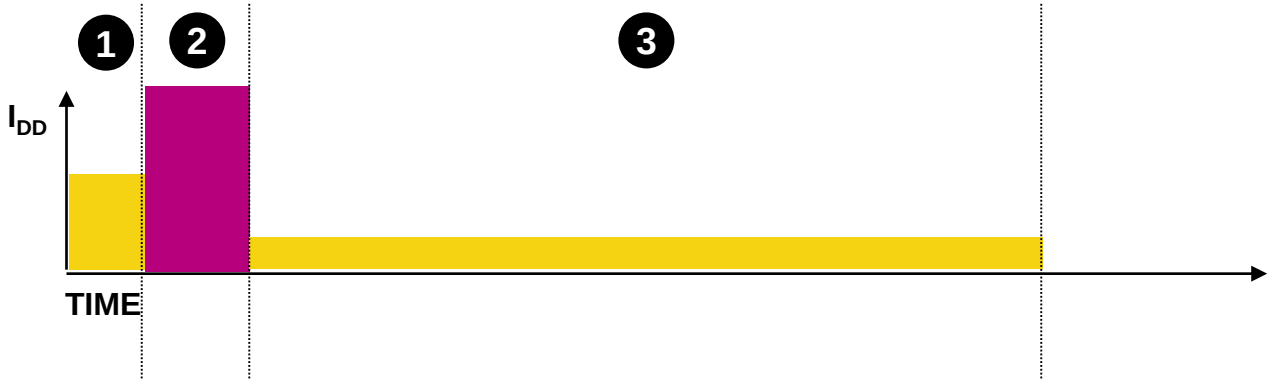
Scenario #1

1 SAMPLING

2 PROCESSING

3 SENDING





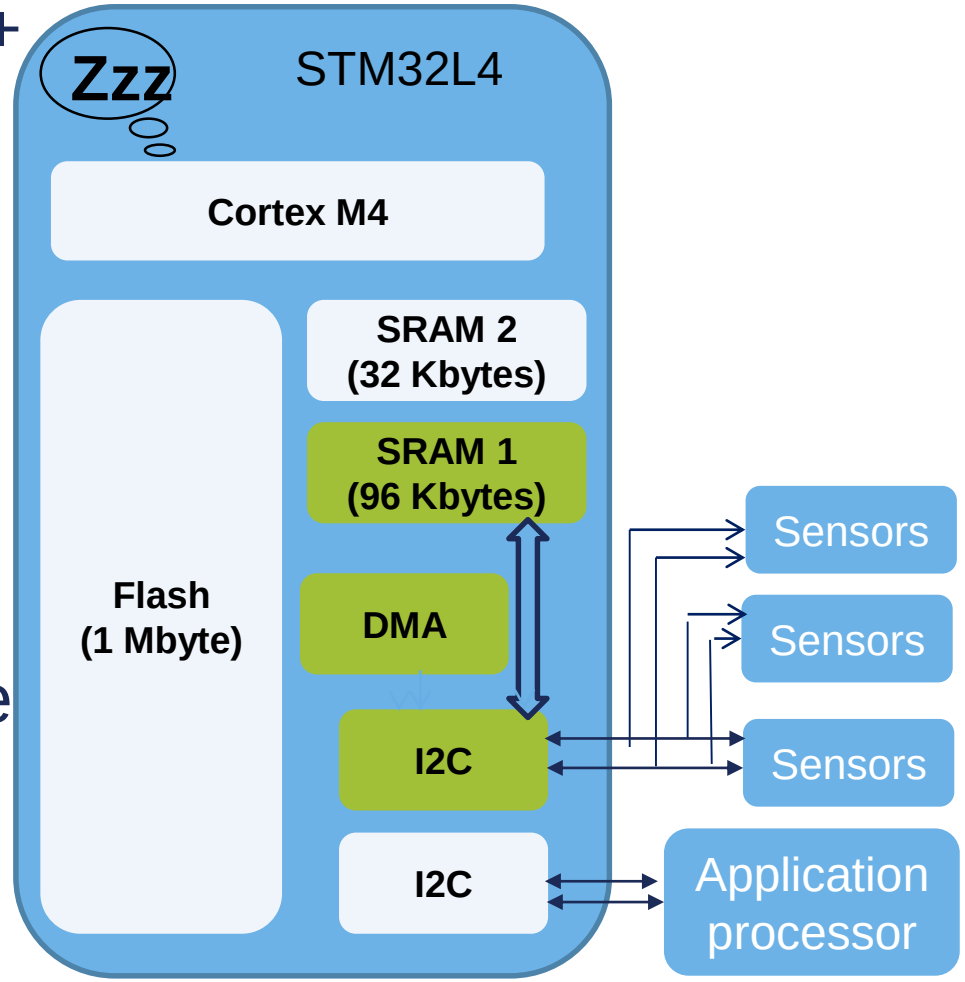
Batch Acquisition mode (BAM)

Optimized mode for transferring data with communication peripherals, while the rest of the device is in low power.

1. Only the needed communication peripheral + 1 DMA + 1 SRAM are configured with clock enabled in Sleep mode

2. Flash memory is put in Power-down mode and Flash clock is gated off during Sleep mode

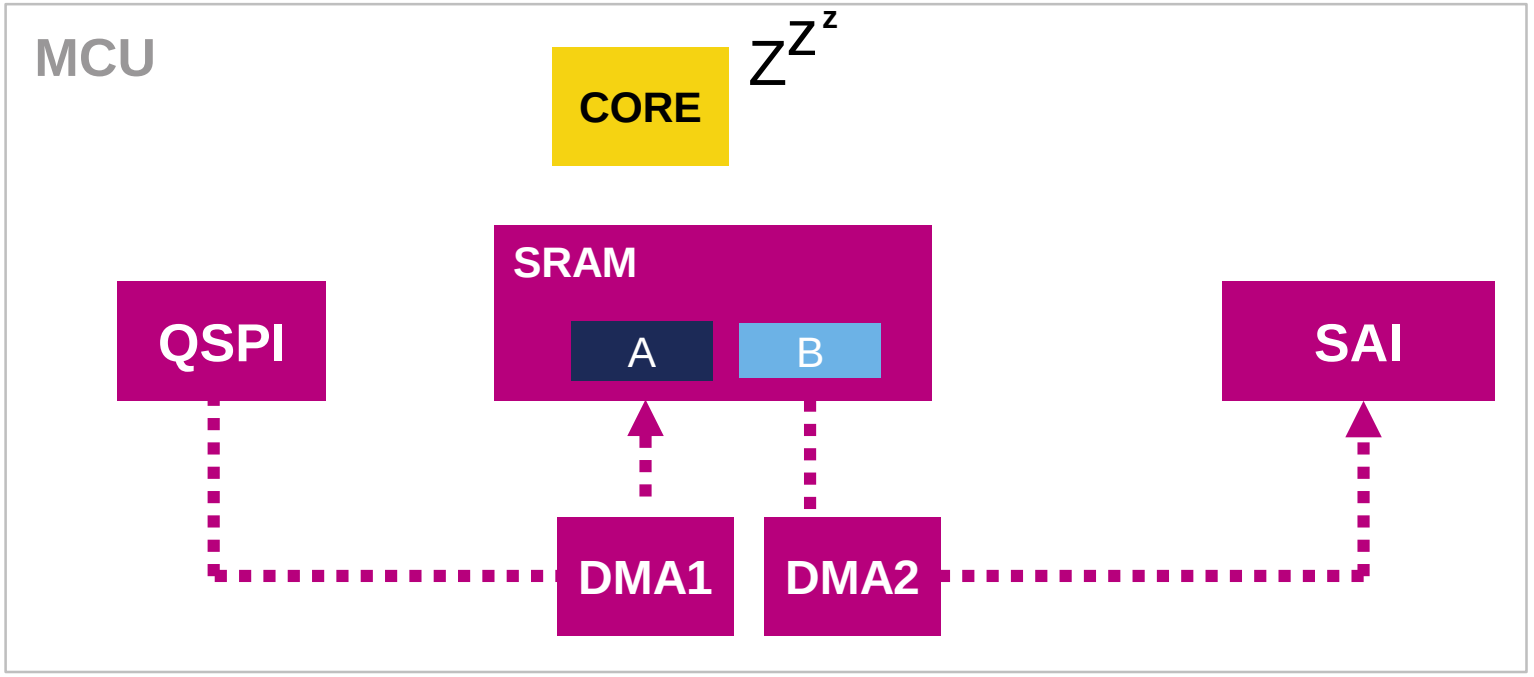
3. Enter either Sleep or Low-power sleep mode
   - Note that the I2C clock can be at 16 MHz even in Low-power sleep mode, allowing 1 MHz Fast-mode Plus support. U(S)ART/LPUART clock can also be HSI.

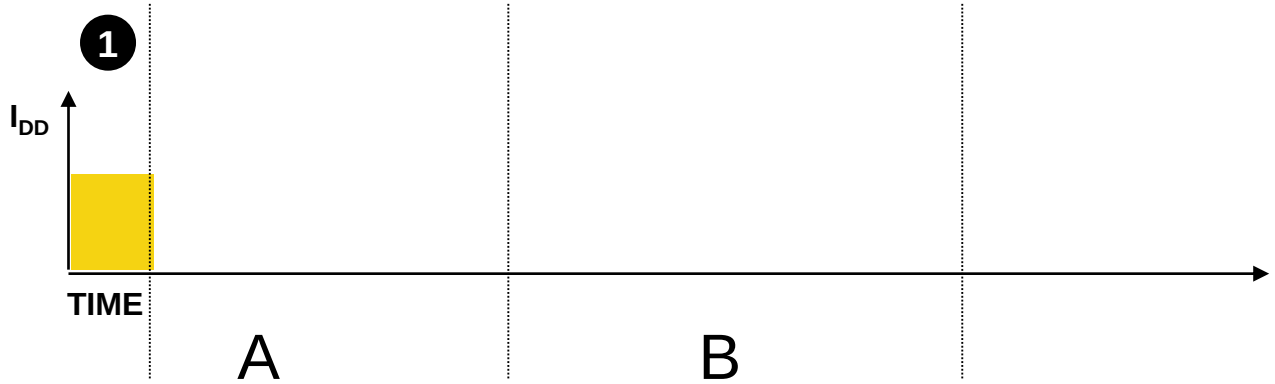


Let the peripherals do the job

Scenario #2

1
Reading new data



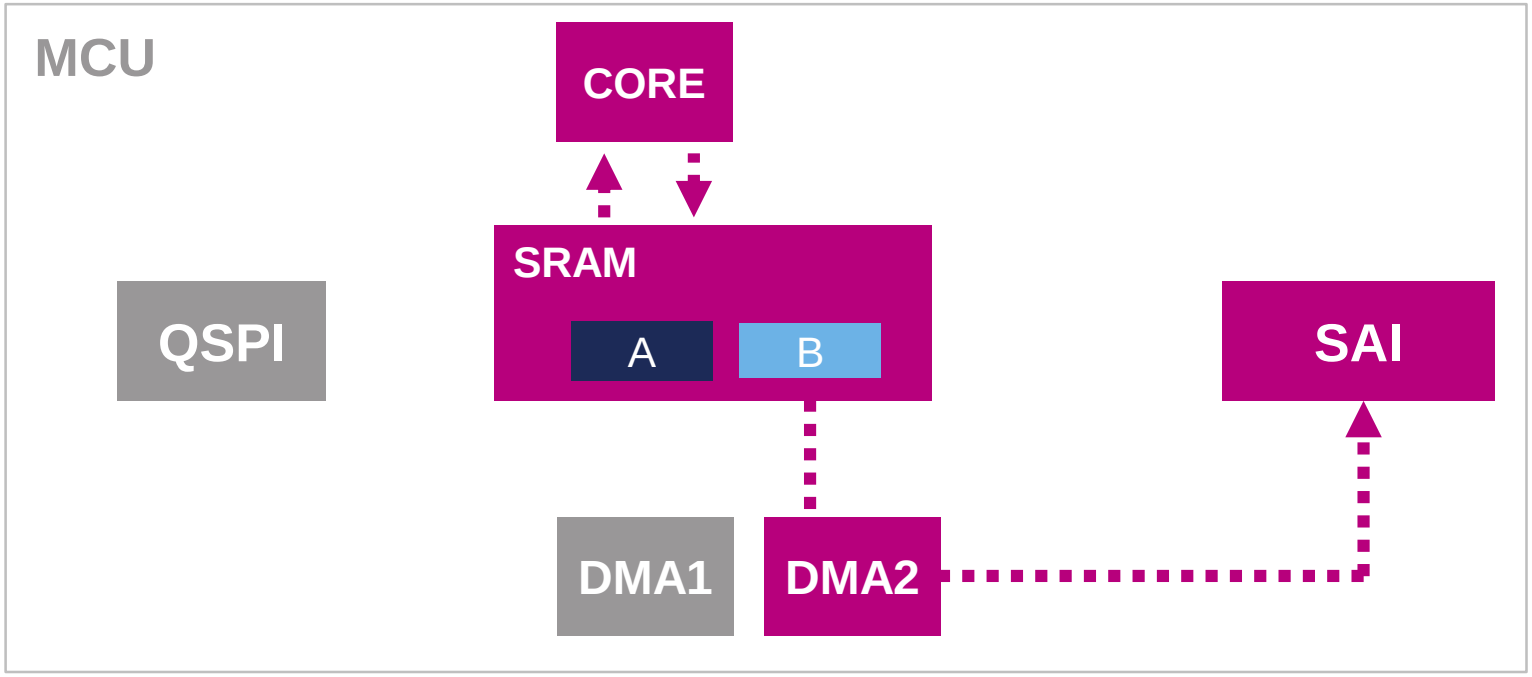


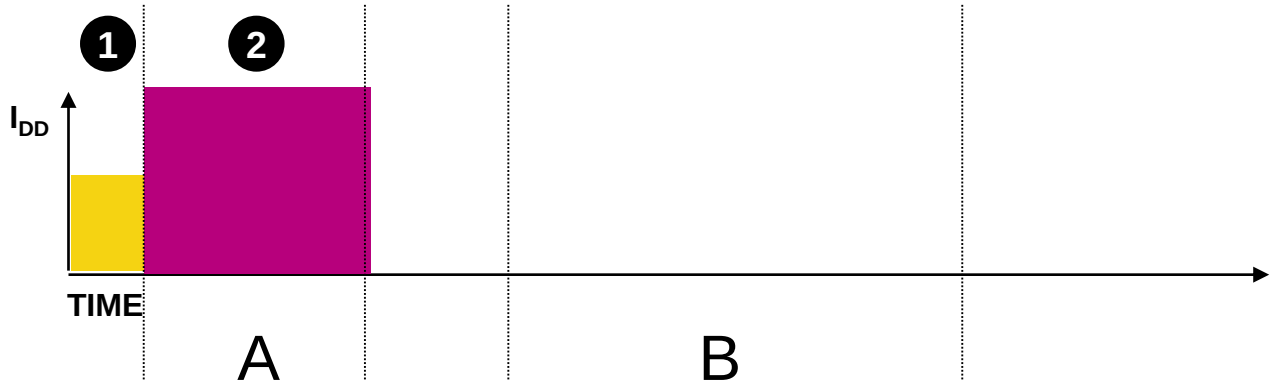
Let the peripherals do the job

Scenario #2

1
Reading new data

2
Processing





life.augmented

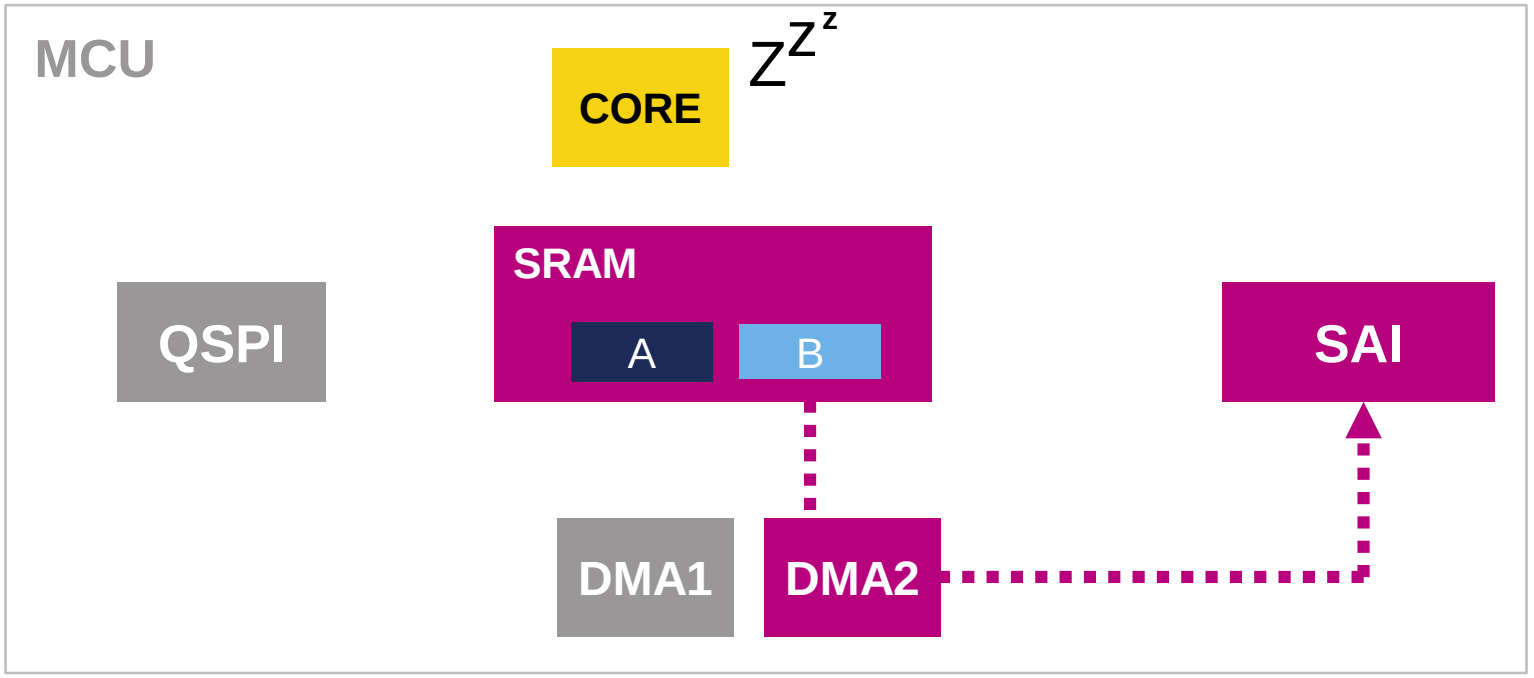
Let the peripherals do the job

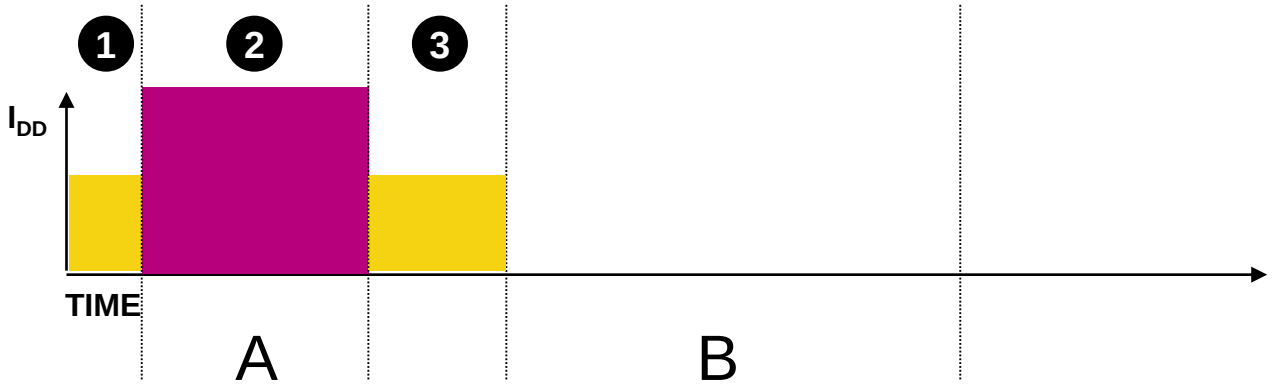
Scenario #2

1 Reading new data

2 Processing

2 Idling



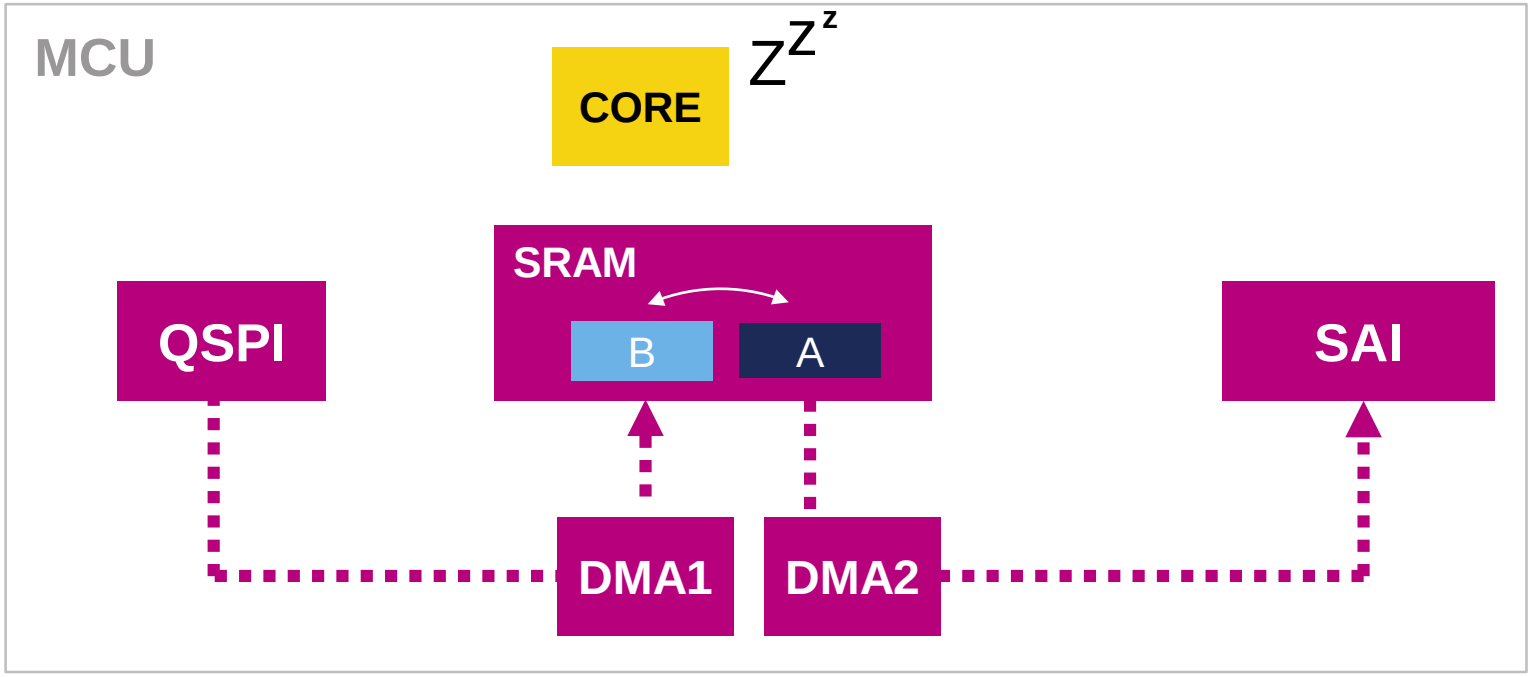


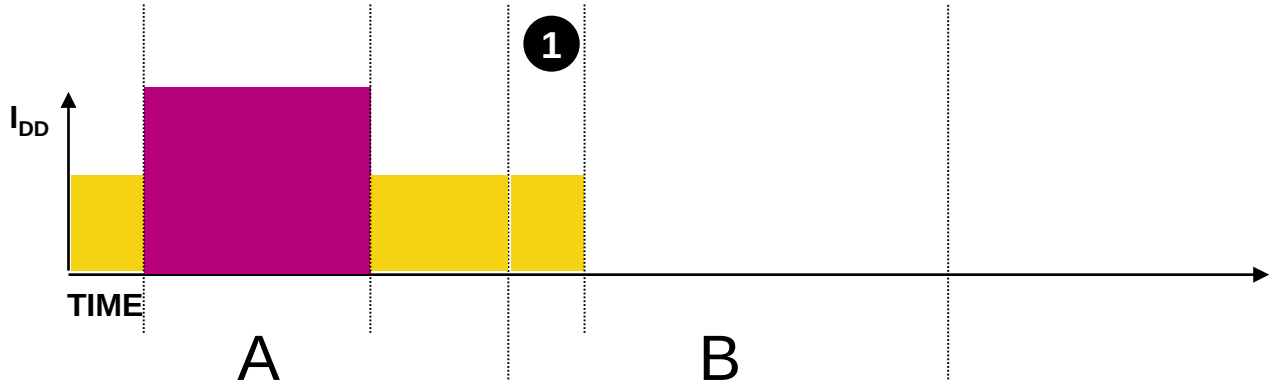
life.augmented

Let the peripherals do the job

Scenario #2

1
Reading new data



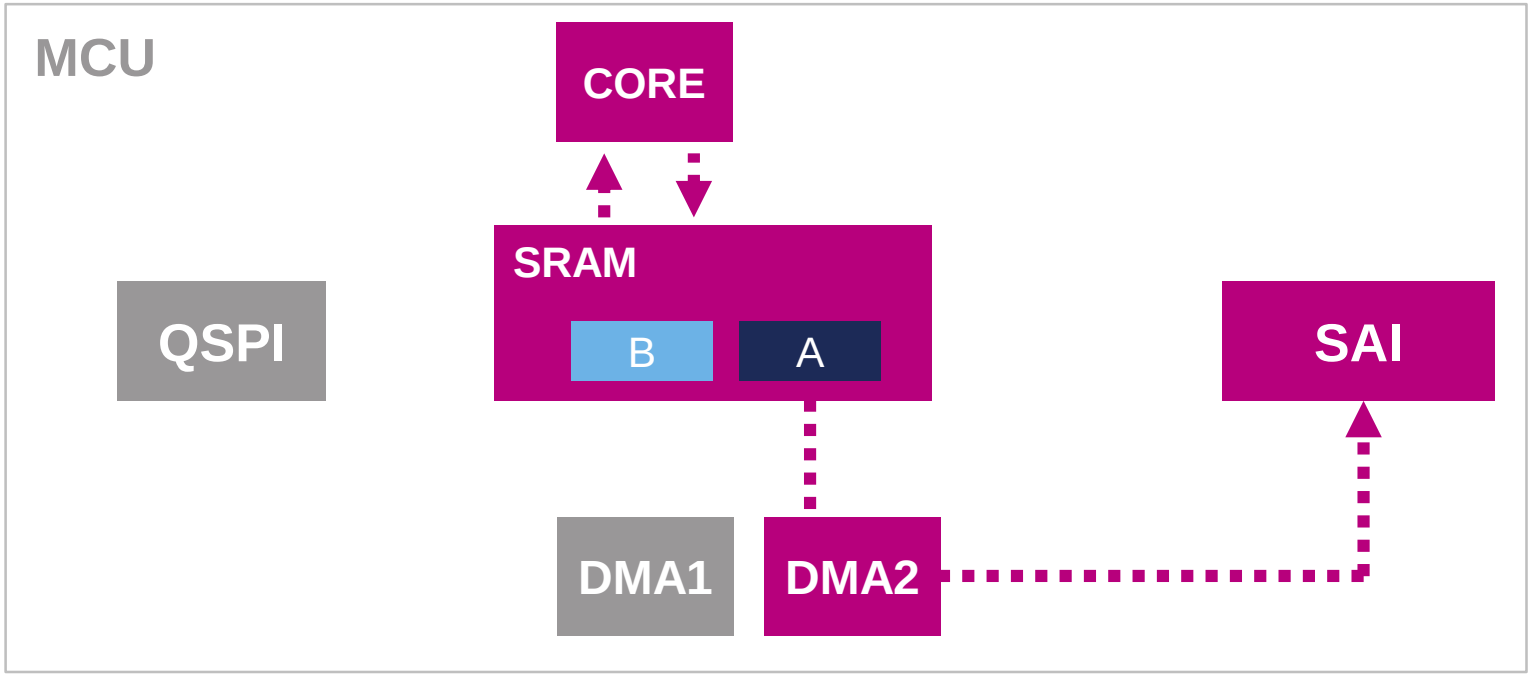


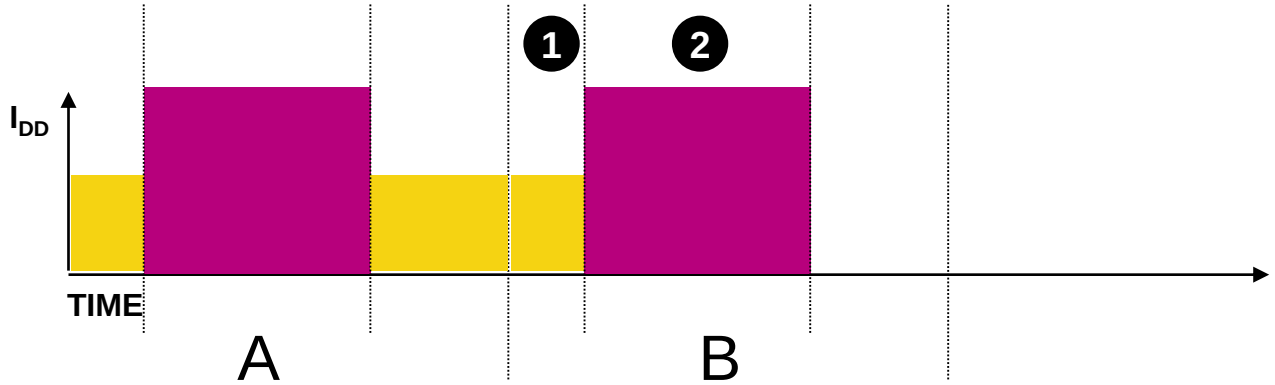
Let the peripherals do the job

Scenario #2

1
Reading new data

2
Processing





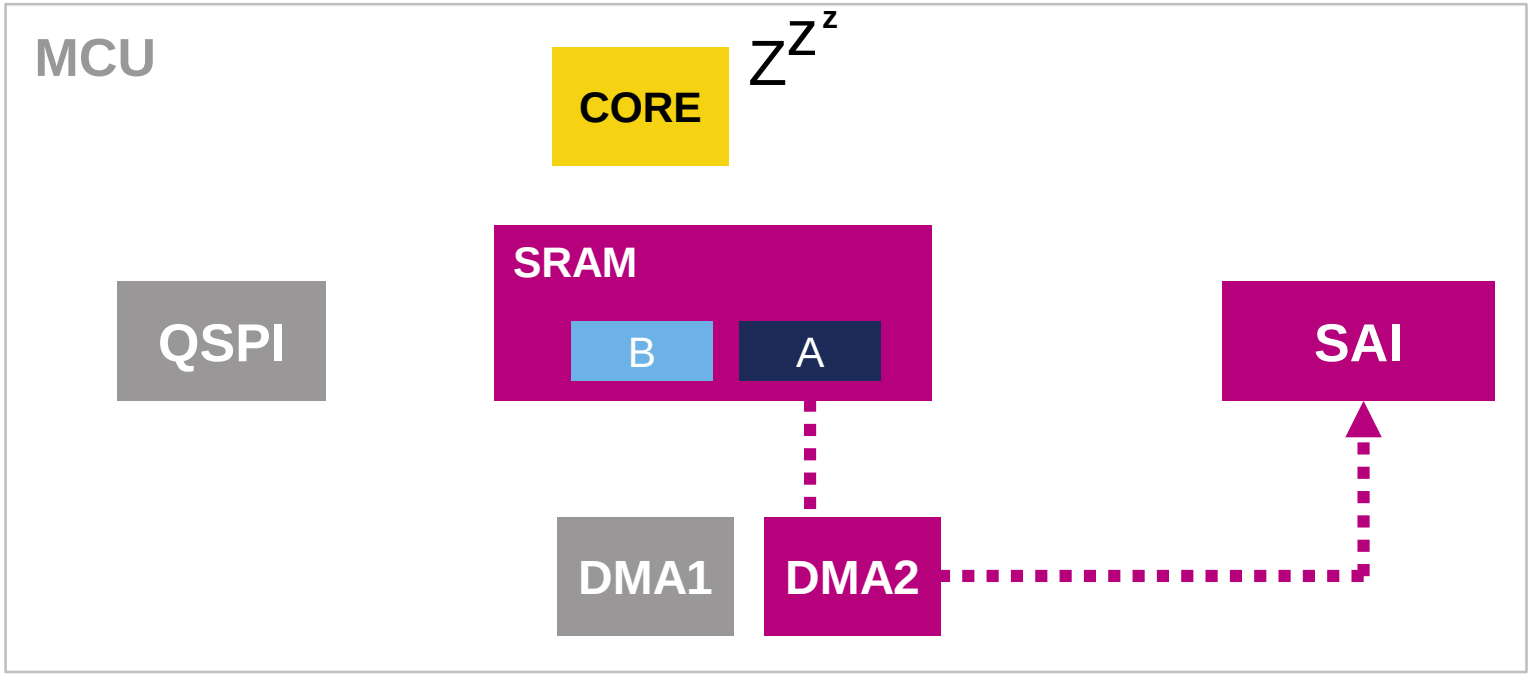
Let the peripherals do the job

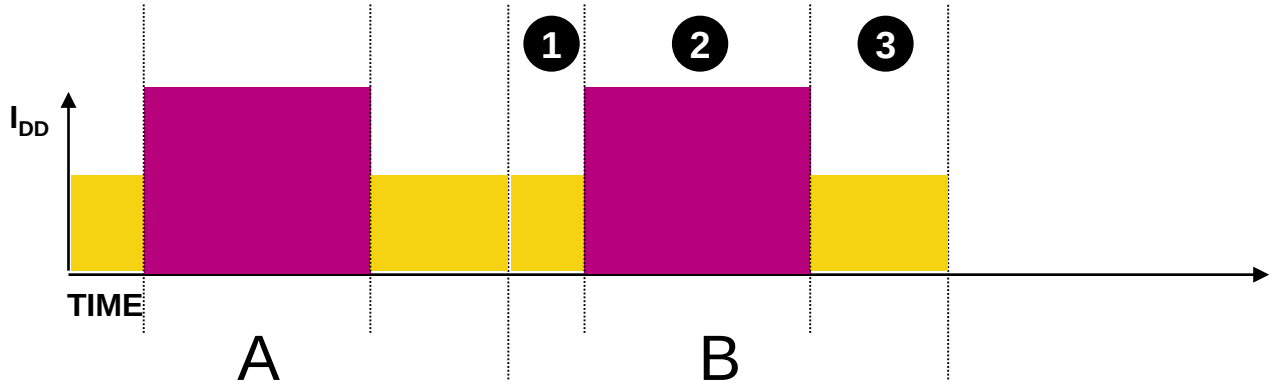
Scenario #2

1 Reading new data

2 Processing

3 Idling

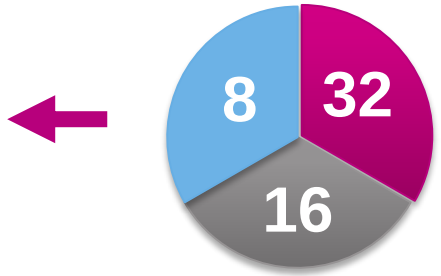




life.augmented

Squeeze the Maximum

1. From the architecture



2. From the peripherals



3. From the compiler

GCC optimizations options

`-O1`

Optimize. Optimizing compilation takes somewh

With `-O`, the compiler tries to reduce code size an

`-O` turns on the following optimization flags:

```
-fauto-inc-dec
-fbranch-count-reg
-fcombine-stack-adjustments
-fcompare-elim
-fcprop-registers
-fdce
-fdefer-pop
-fdelayed-branch
-fdse
-fforward-propagate
-fguess-branch-probability
-fif-conversion2
-fif-conversion
-finline-functions-called-once
-fipa-pure-const
-fipa-profile
-fipa-reference
-fmerge-constants
-fmove-loop-invariants
-freorder-blocks
-fshrink-wrap
-fsplit-wide-types
-ftree-bit-ccp
-ftree-ccp
-fssa-phiopt
-ftree-ch
-ftree-coalesce-vars
-ftree-copy-prop
-ftree-dce
-ftree-dominator-opts
-ftree-dse
-ftree-forwprop
-ftree-fre
-ftree-phiprop
-ftree-sink
-ftree-slsr
-ftree-sra
-ftree-pta
-ftree-ter
-funit-at-a-time
```

`-O2` turns on all optimization flags specified by `-O`. It also turns on the following optimization flags:

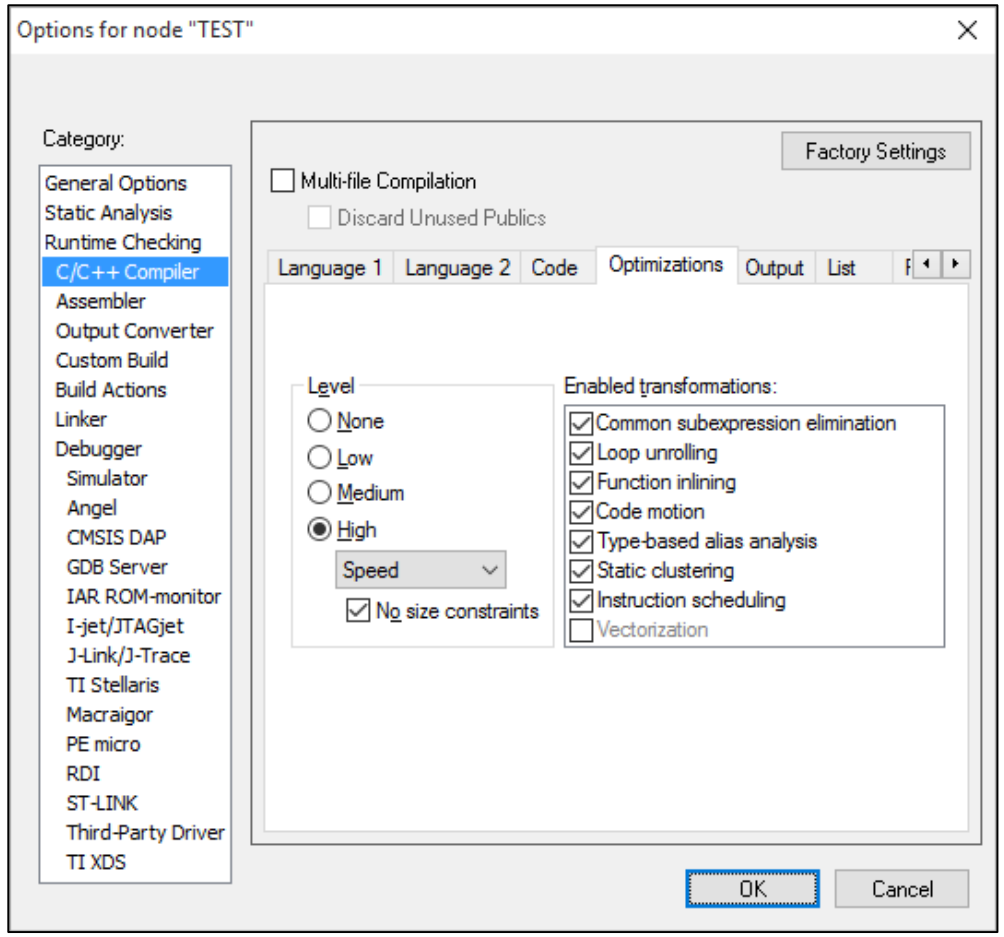
```
-fthread-jumps
-falign-functions  -falign-jumps
-falign-loops  -falign-labels
-fcaller-saves
-fcrossjumping
-fcse-follow-jumps  -fcse-skip-blocks
-fdelete-null-pointer-checks
-fdevirtualize -fdevirtualize-speculatively
-fexpensive-optimizations
-fgcse  -fgcse-lm
-fhoist-adjacent-loads
-finline-small-functions
-findirect-inlining
-fipa-cp
-fipa-cp-alignment
-fipa-sra
-fipa-icf
-fisolate-erroneous-paths-dereference
-flra-remat
-foptimize-sibling-calls
-foptimize-strlen
-fpartial-inlining
-fpeephole2
-freorder-blocks-algorithm=stc
-freorder-blocks-and-partition -freorder-functions
-frerun-cse-after-loop
-fsched-interblock  -fsched-spec
-fschedule-insns  -fschedule-insns2
-fstrict-aliasing -fstrict-overflow
-ftree-builtin-call-dce
-ftree-switch-conversion -ftree-tail-merge
-ftree-pre
-ftree-vrp
-fipa-ra
```

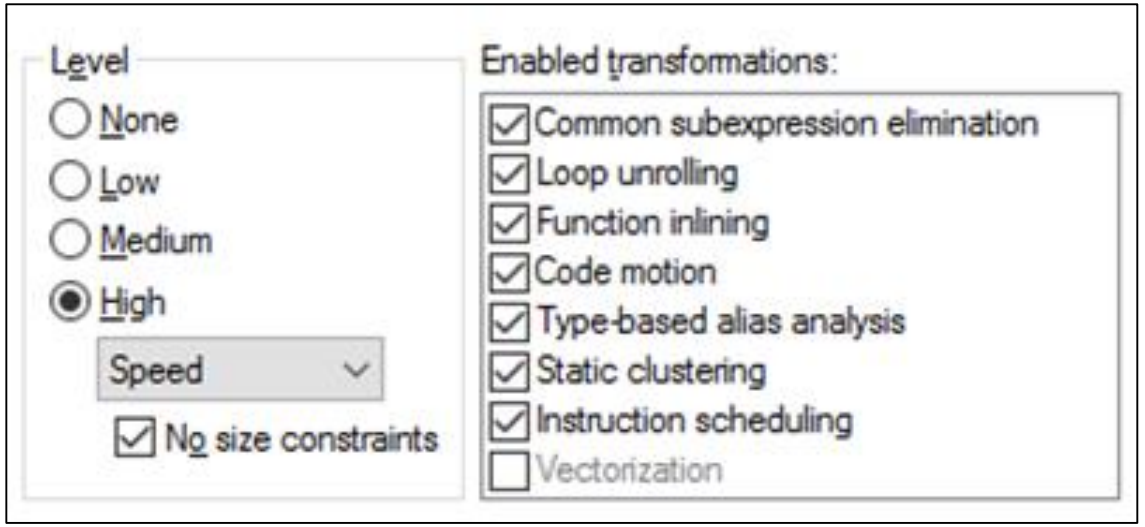
`-O3`

Optimize yet more. `-O3` turns on all optimizations specified by `-O2` and also turns on the `-finline-functions`, `-funswitch-loops`, `-fpredictive-commoning`, `-fgcse-after-reload`, `-ftree-loop-vectorize`, `-ftree-loop-distribute-patterns`, `-ftree-slp-vectorize`, `-fvect-cost-model`, `-ftree-partial-pre` and `-fipa-cp-clone` options.

http://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

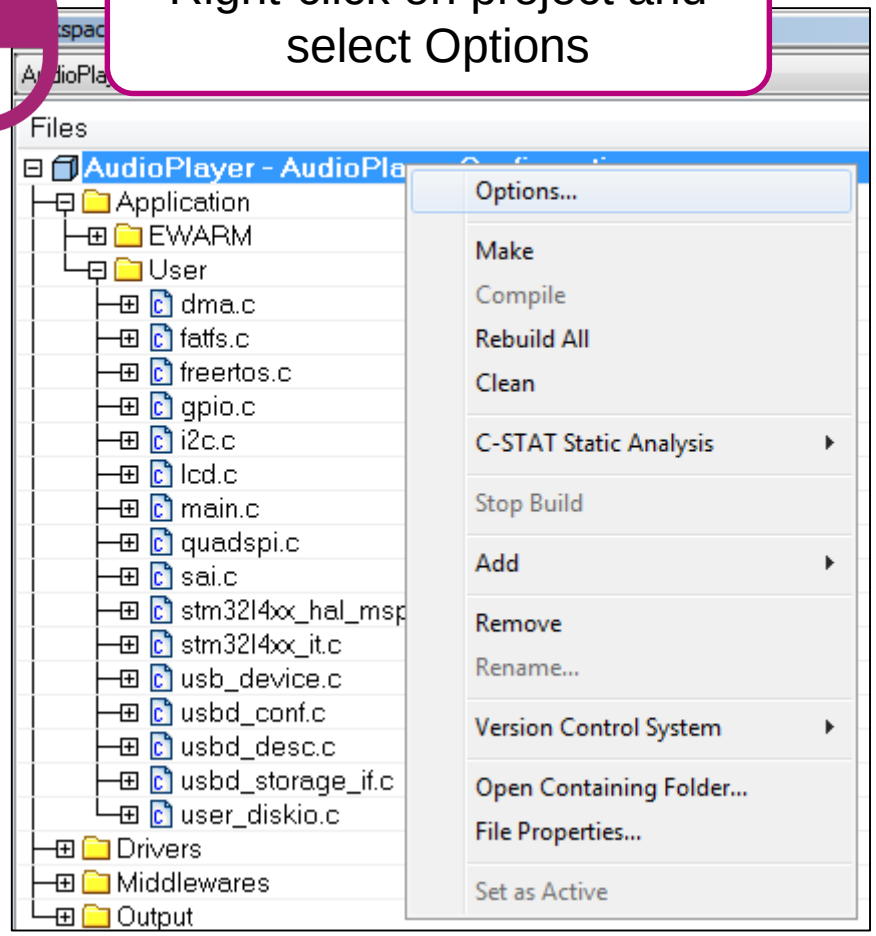
IAR optimization options





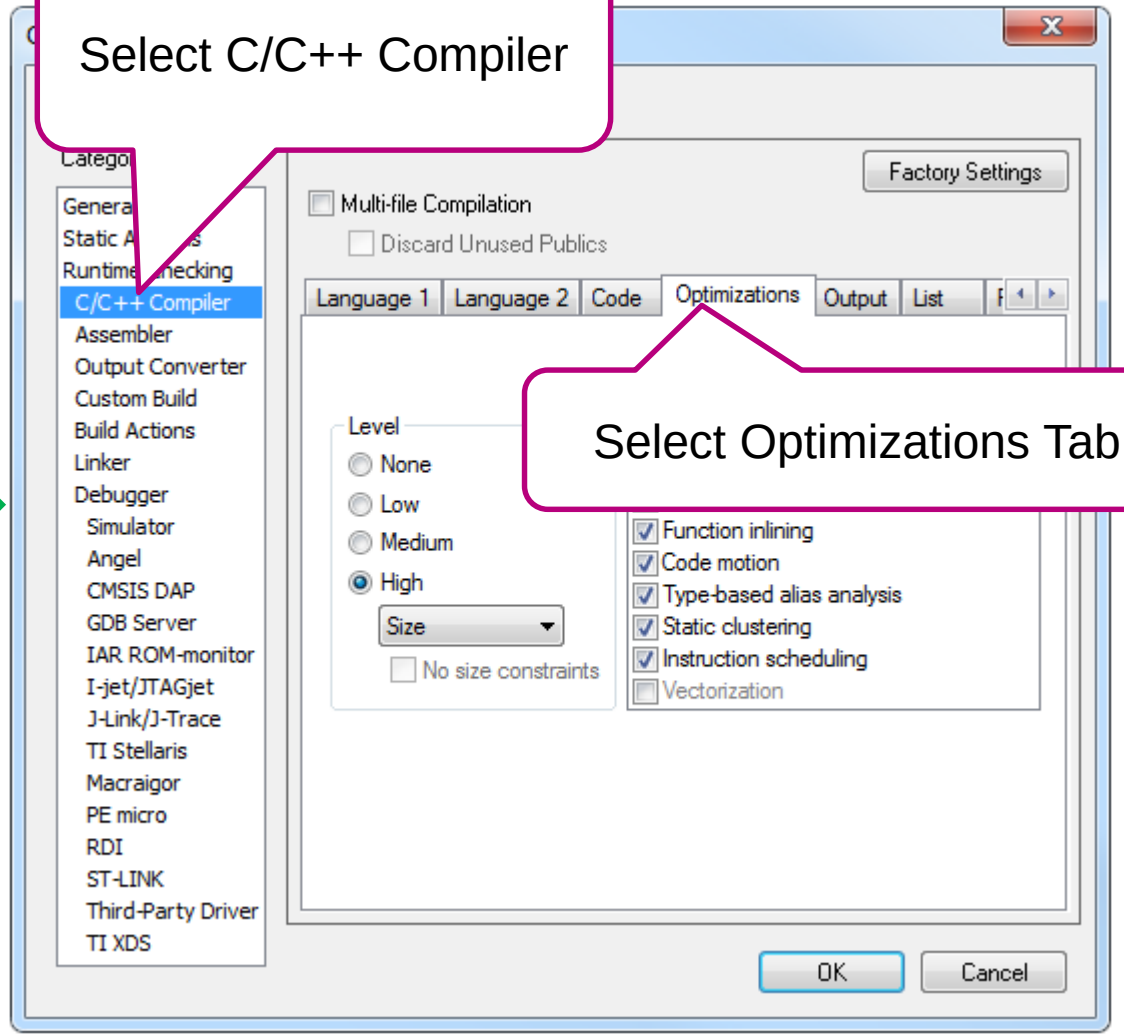
Compiler optimizations and AudioPlayer

Right-click on project and select Options



Select C/C++ Compiler

Select Optimizations Tab



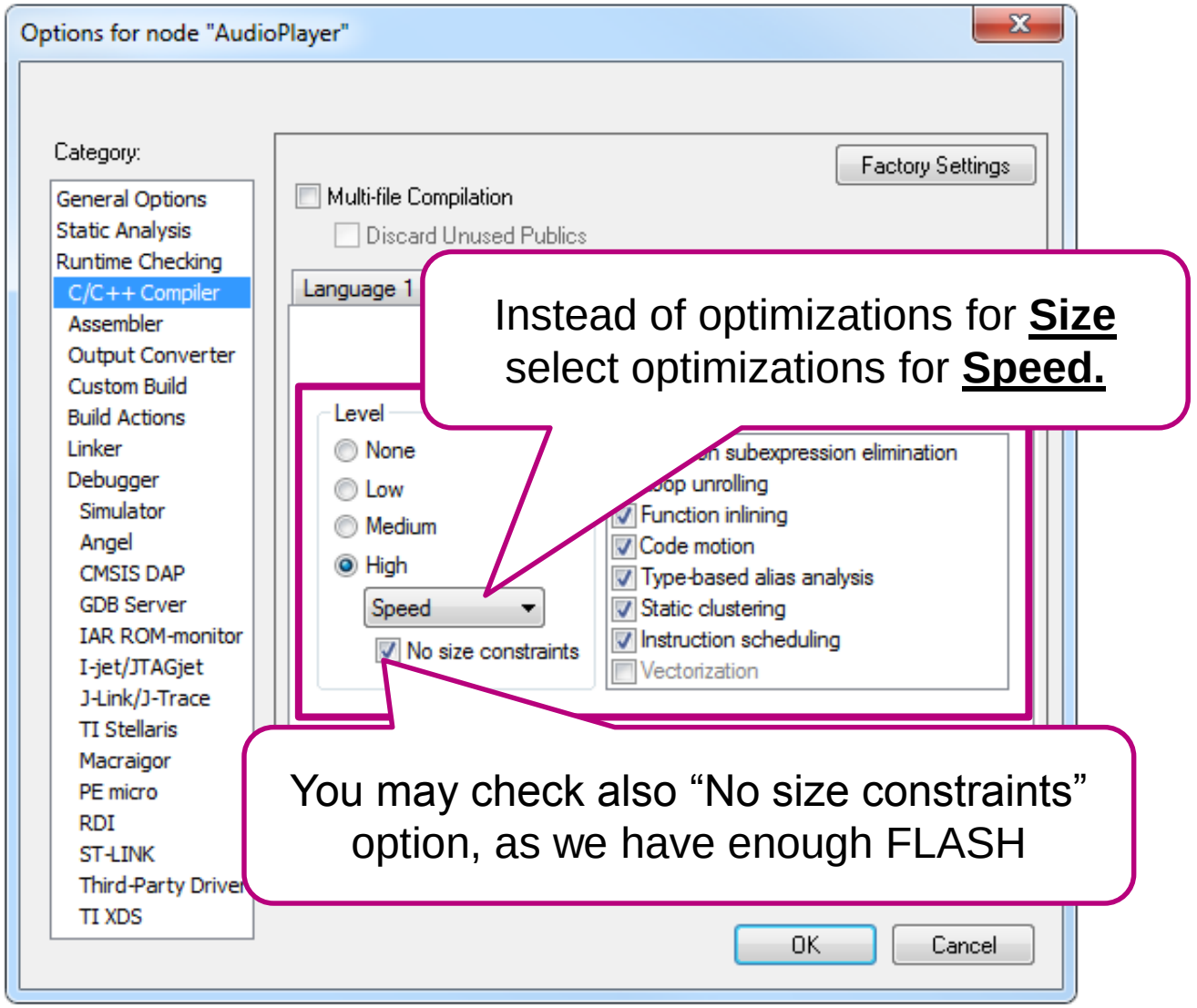
life.augmented

4 Let’s try it out

1. Open the AudioPlayer IAR project (from previous step)
2. Open project options window from the menu Project->Options
3. Select C/C++ Compiler category
4. Select Optimization tab
5. Select “Speed” option
6. Recompile the code, download to the board
7. Check the code size and measure current consumption in PLAY and STOP modes of the player.

life.augmented

Compiler optimizations and AudioPlayer



Compiler optimizations and AudioPlayer

Code size impact

optimization	code size
max speed	…….. B
min size	…….. B

life.augmented

Compiler optimizations and AudioPlayer

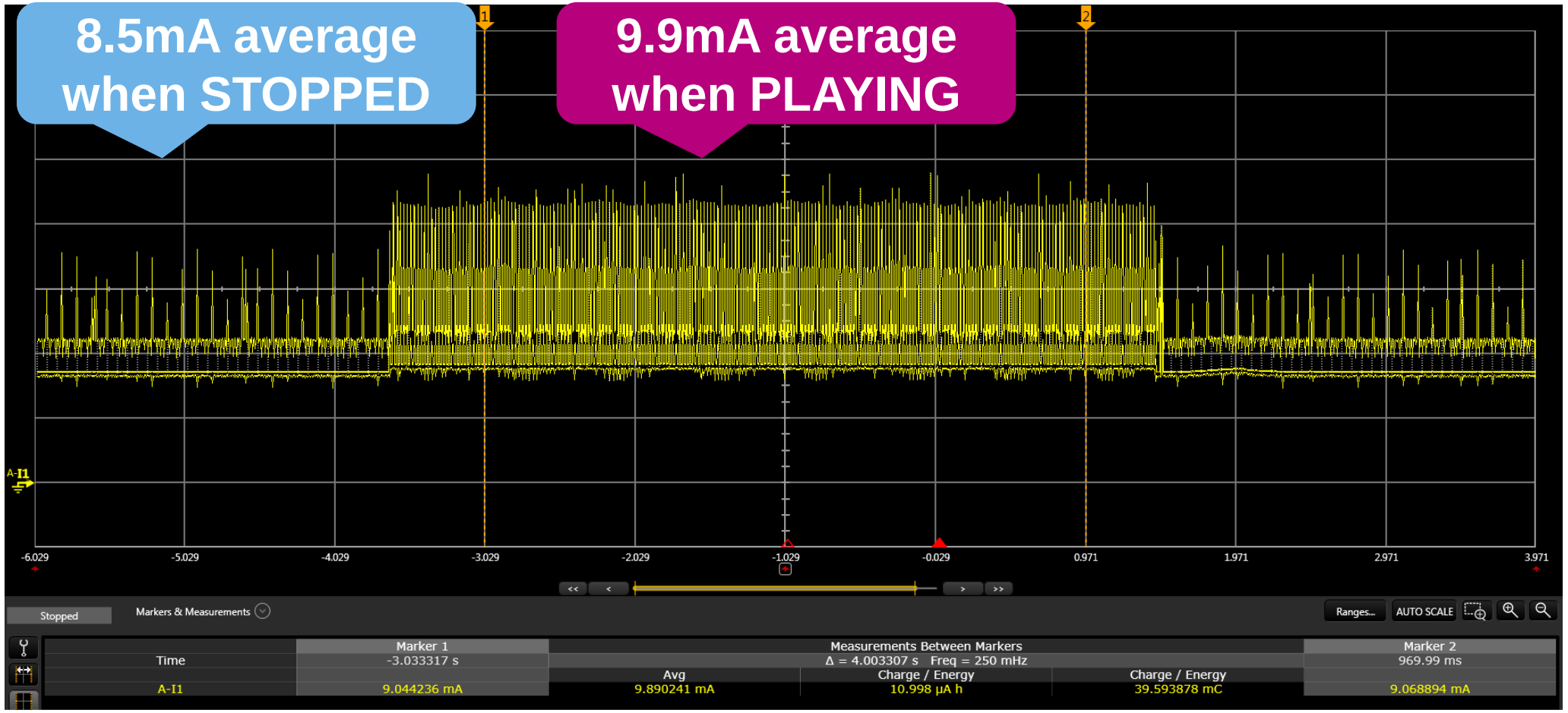
Code size impact

optimization	code size
max speed	72 652 B
min size	43 184 B

life.augmented

AudioPlayer optimization 4/5

Compiler settings (optimization for speed selected)



life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0
Compiler settings	….. mA	….. 0 %	….. mA	….. %

Just the MCU consumption considered
(including circuitries supplied from GPIOs)

life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0
Compiler settings	8.5 mA	~0	9.9 mA	-6.6%

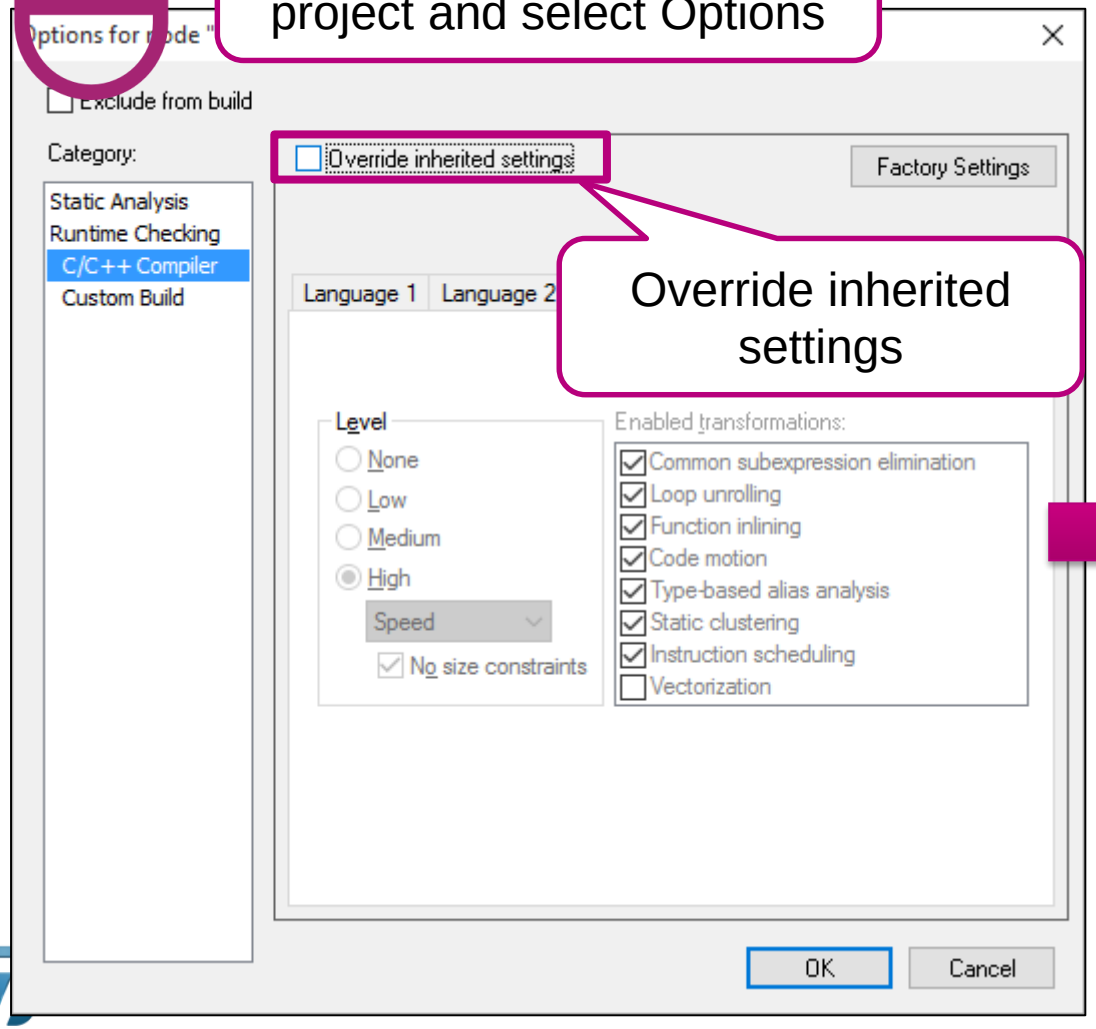
Just the MCU consumption considered
(including circuitries supplied from GPIOs)

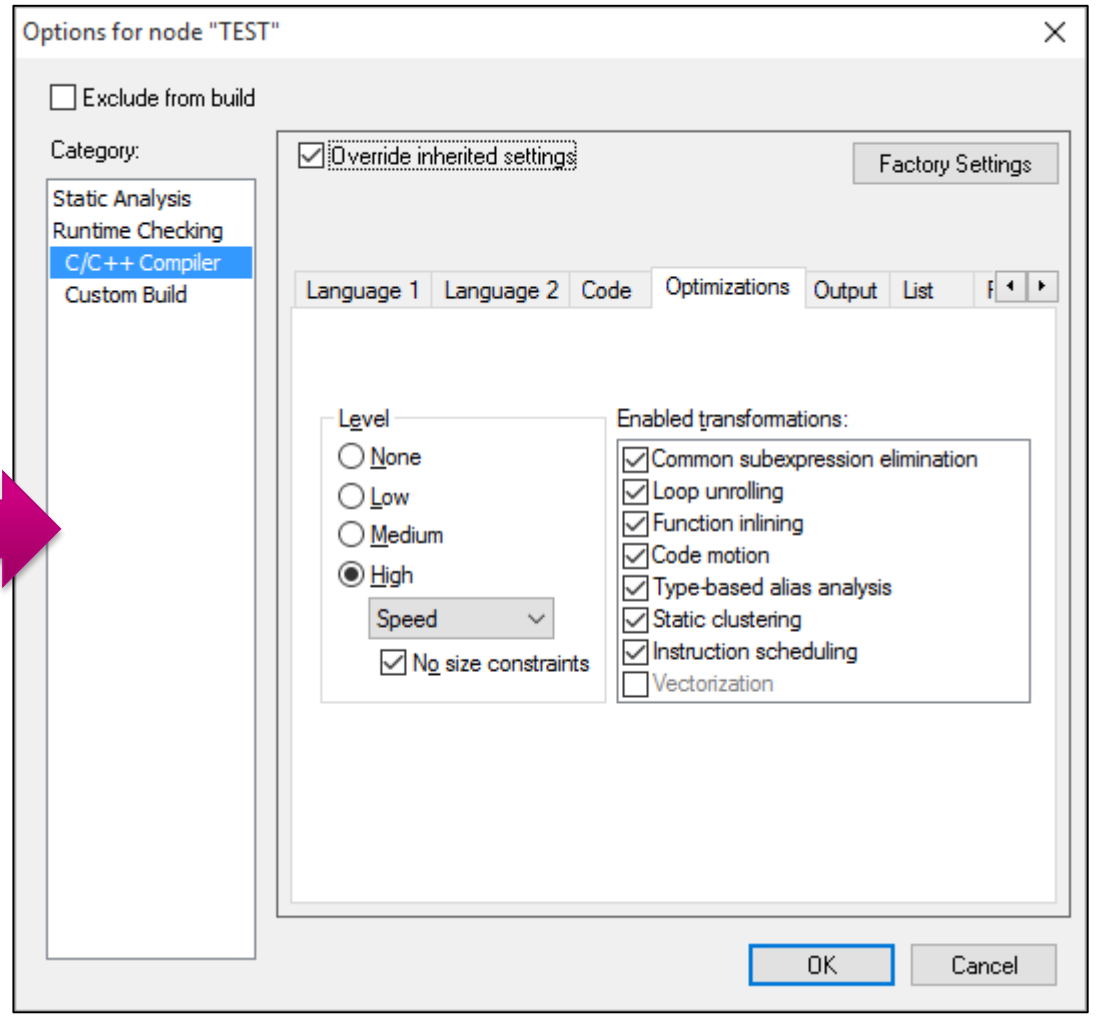
life.augmented

Optimizations Level per file

IAR (EXAMPLE)

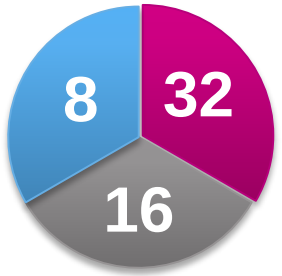
Right-click on desired file in project and select Options





Squeeze the Maximum

1. From the architecture

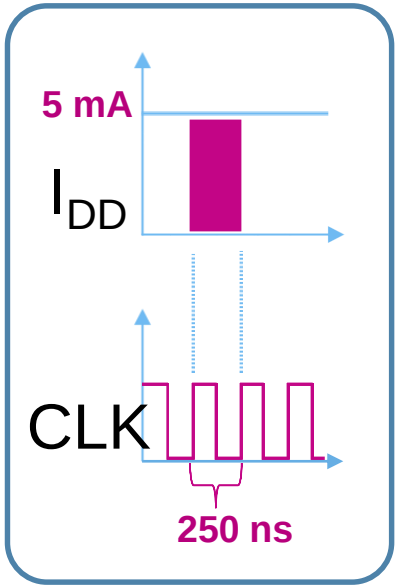


2. From the peripherals

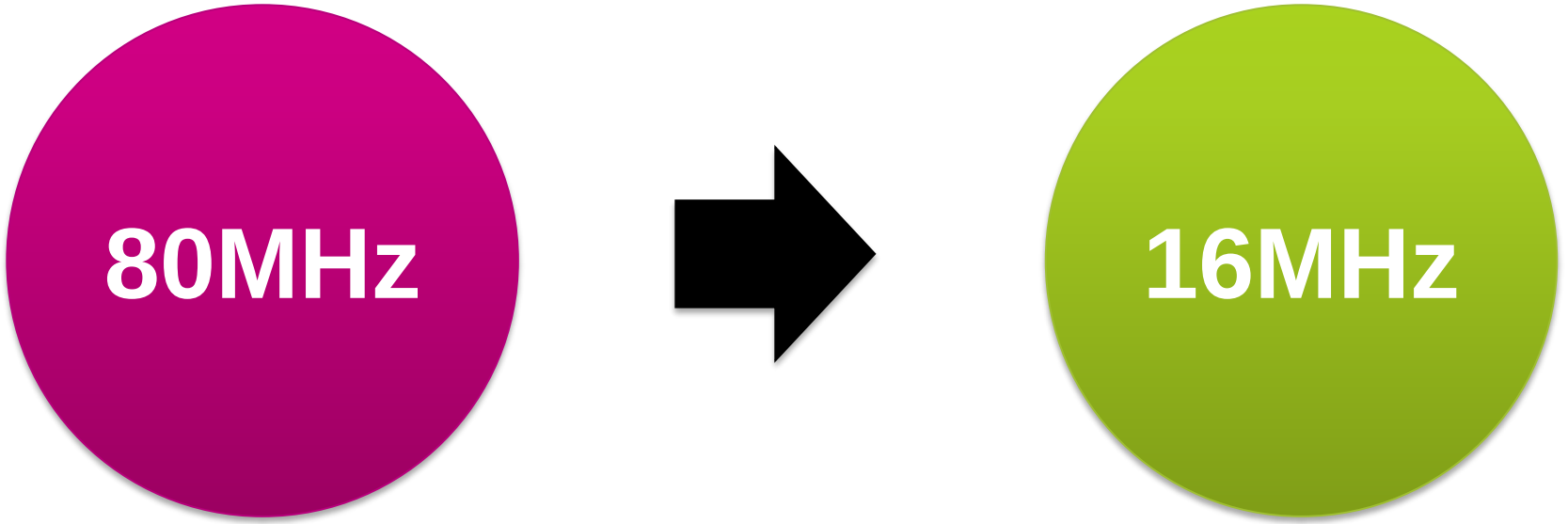


3. From the compiler

4. From every clock cycle



Why do we run @80MHz?



Let’s slow down, just PLL settings to be slightly modified

We can do it within Clock Configuration tab in CubeMX project.
Please keep 48MHz for USB and 11.294118 MHz for SAI1

5 Let’s try it out

1. Open the CubeMX project (from previous step)
2. Go to Clock Configuration tab
3. Change clock configuration of the system clock from 80MHz to 16MHz

   keeping 48MHz for USB and 11.29MHz for SAI1
4. Generate the code, apply Patch and compile it
5. Connect discovery kit over miniUSB cable (ST-Link)
6. Download the code to the STM32L4Discovery
7. Press the reset button on discovery once download has finished

Apply small patch

- Apply patch from folder
- **C:\STM32L4_Workshop\2017_HandsOn\HandsOn\3_Tips_and_Tricks\Patch** to project (C:\AudioPlayer\AudioPlayer\)
- It contains the following files:

  .\Src\

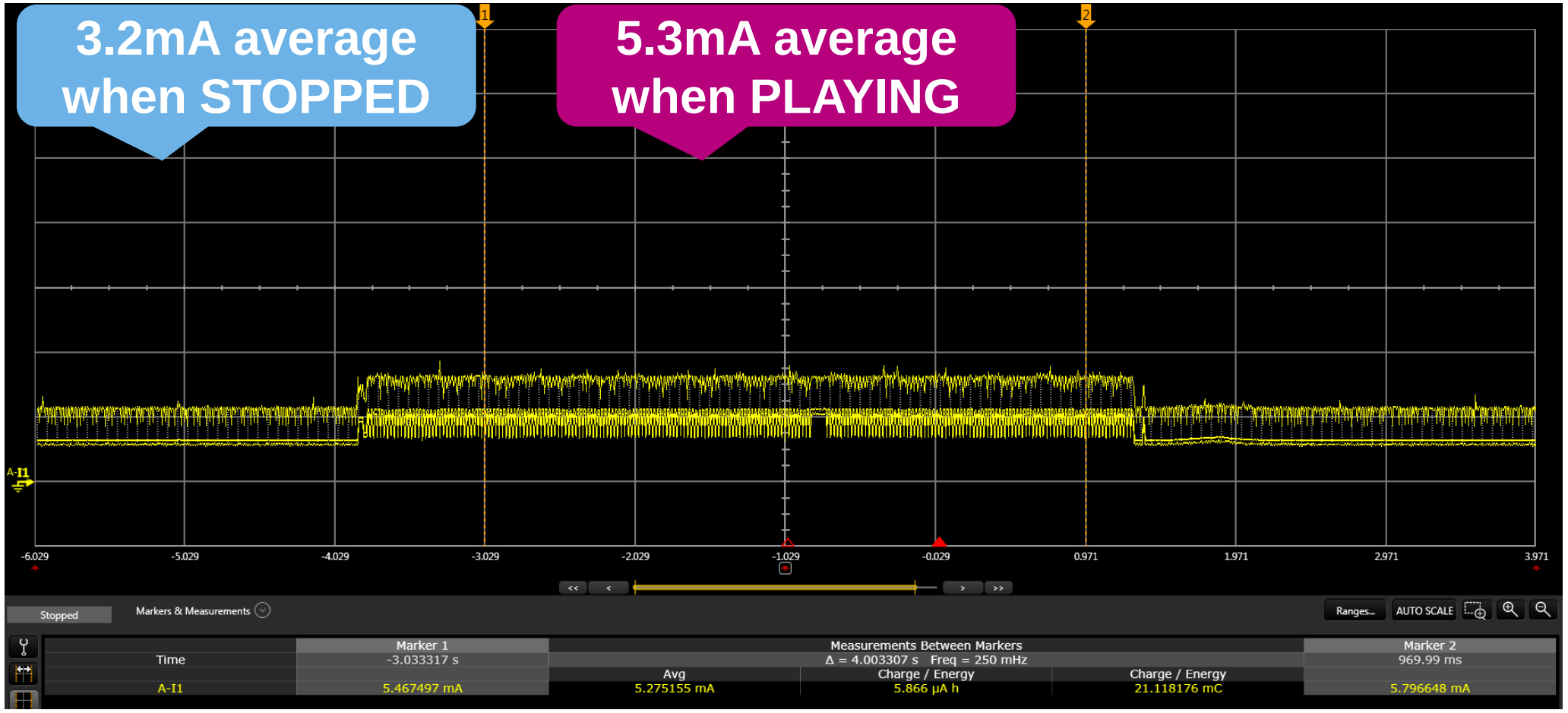
  user_diskio.c

  sai.c

This operation is STM32CubeMX non-intrusive. You can re-generate the project later and the modifications will be retained. (except sai.c and user_diskio.c)

AudioPlayer optimization 5/5

Slower clocks



AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0
Compiler settings	8.5 mA	~0	9.9 mA	-6.6%
Slower clocks	….. mA	….. %	….. mA	….. %

Just the MCU consumption considered
(including circuitries supplied from GPIOs)

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0
Compiler settings	8.5 mA	~0	9.9 mA	-6.6%
Slower clocks	3.2 mA	-62.3%	5.3 mA	-46.5%

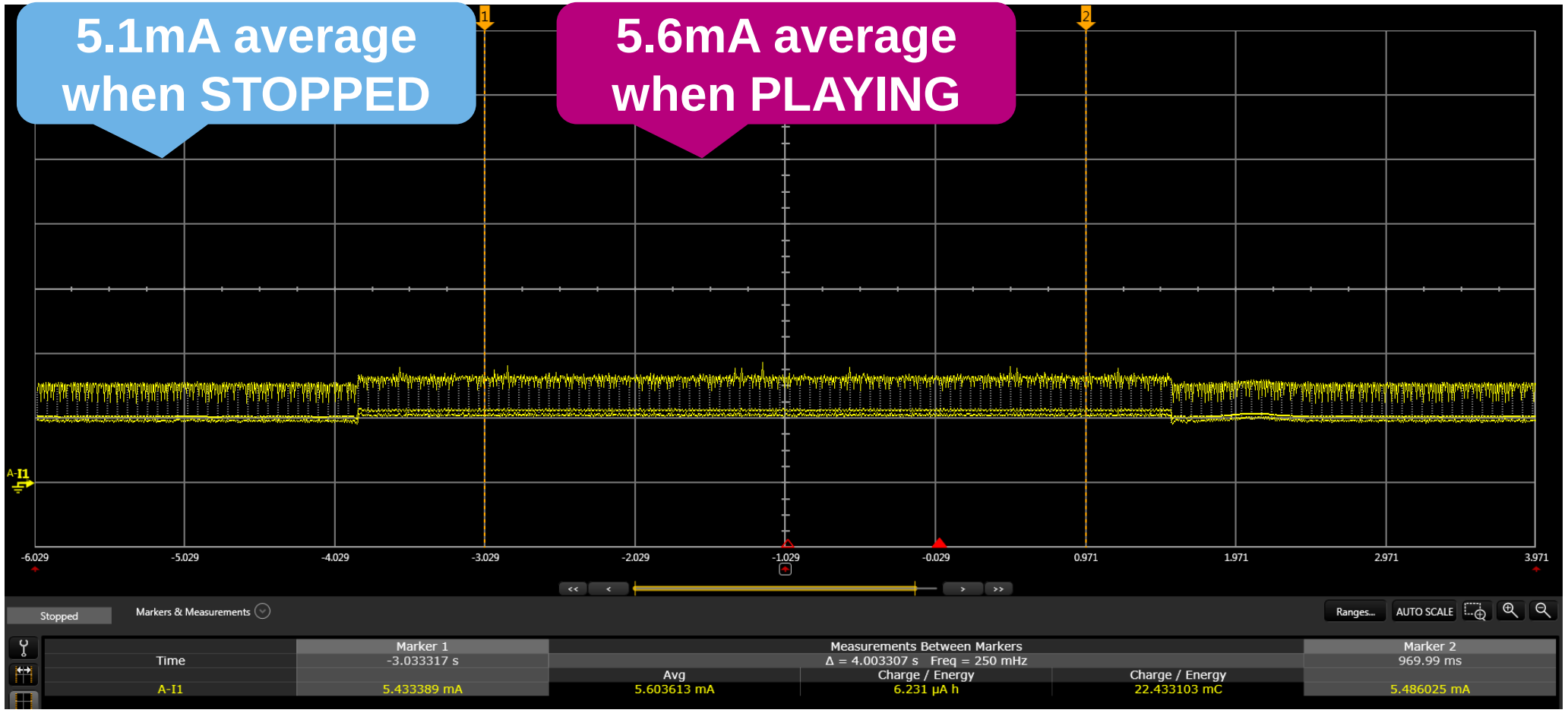
Just the MCU consumption considered
(including circuitries supplied from GPIOs)

AudioPlayer optimization 5/5

Slower clocks (without Tickless Mode)

5.1mA average when STOPPED

5.6mA average when PLAYING



life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0
Compiler settings	8.5 mA	~0	9.9 mA	-6.6%
Slower clocks	3.2 mA	-62.3%	5.3 mA	-46.5%
	….. mA	….. %	….. mA	….. %

Tickless Mode disabled

life.augmented

AudioPlayer optimization summary

V_{DD}=3.0V @25°C

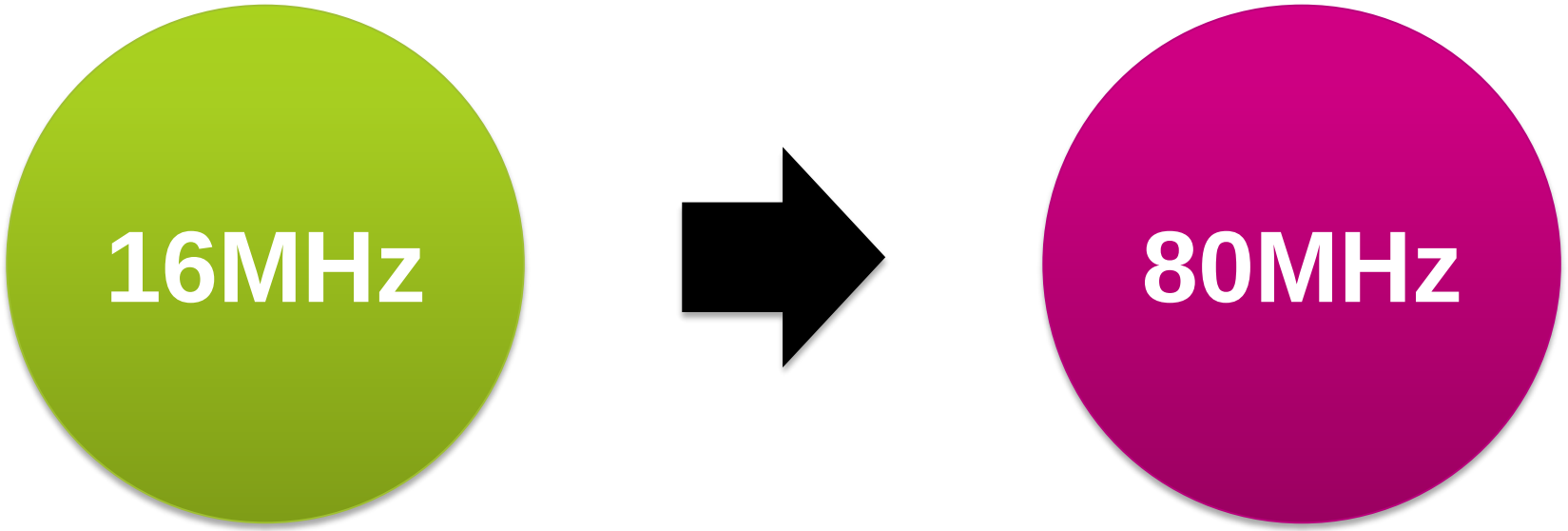
Optimization applied	STOPPED		PLAYING	
	I_{DD}	Difference	I_{DD}	Difference
Original state (STEP6)	17.3 mA	-	17.8 mA	-
Tickless Mode	10.2 mA	-41%	11.9 mA	-33.1%
Clocks gating	8.5 mA	-16.7%	10.6 mA	-10.9%
Unused GPIOs	8.5 mA	~0	10.6 mA	~0
Compiler settings	8.5 mA	~0	9.9 mA	-6.6%
Slower clocks	3.2 mA	-62.3%	5.3 mA	-46.5%
	5.1 mA	+37.3%	5.6mA	+5.4%

Tickless Mode disabled

life.augmented

High performance on demand

If we would need higher performance (MP3, WMA decoding, DSP post processing, equalizer)

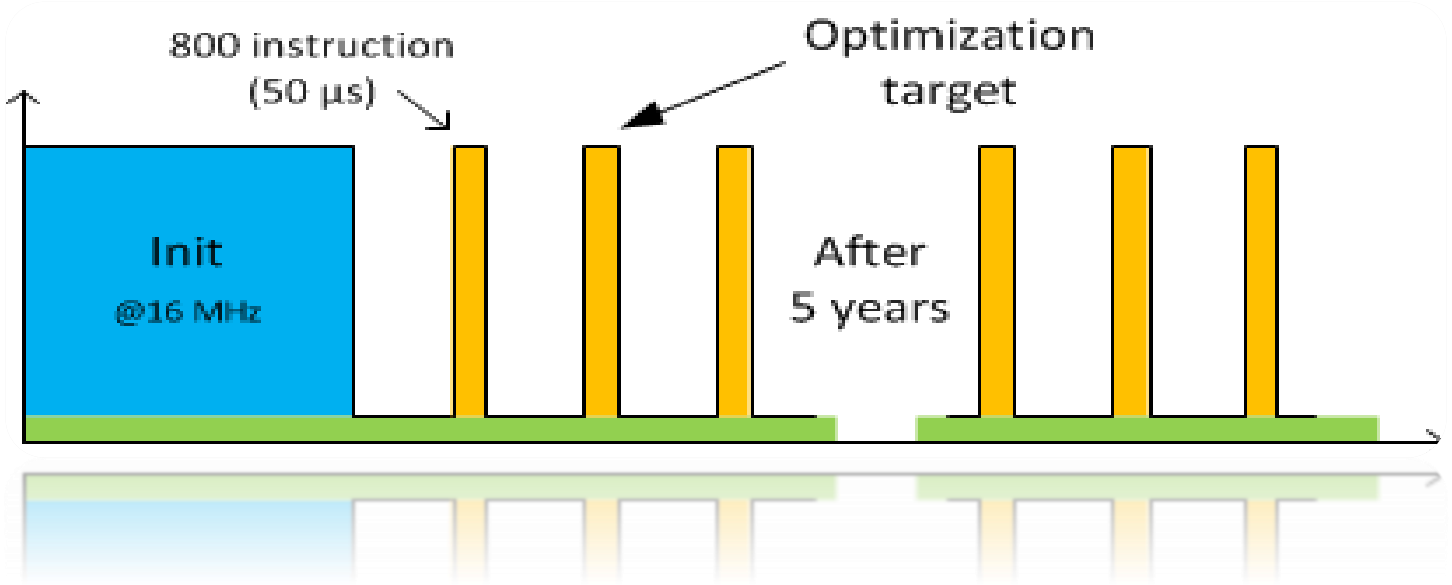


We can switch smoothly in runtime.

Analyze your application!

- Each single instruction is executed 158.10^6 times over the 5years
- Removing only 1 instruction will give **2.3days** time reserve over 5y.

1 instruction removed = **2.3 days saving**



Focus on details

- even 1 extra instruction if optimized (removed) can save a lot of energy

- **Optimized code = Reduced code** with the same functionality

➢ Gain: LOW power consumption + HIGH performance

OPTIMAL CASE !!!

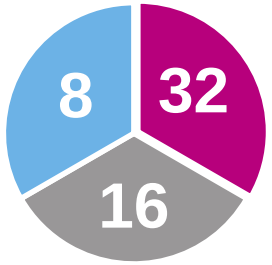
➢ x% of extra instruction = x% waste of core performance

Optimization flow for STM32L4

1. Use low-power modes (at least SLEEP)
2. Use compiler optimizations
3. Re-think use of peripherals
4. Execute from SRAM (real 0WS)
5. Use STM32CubeL4 LL drivers
6. Low level optimizations
7. Increase system frequency only when necessary

Squeeze the Maximum

1. From the architecture

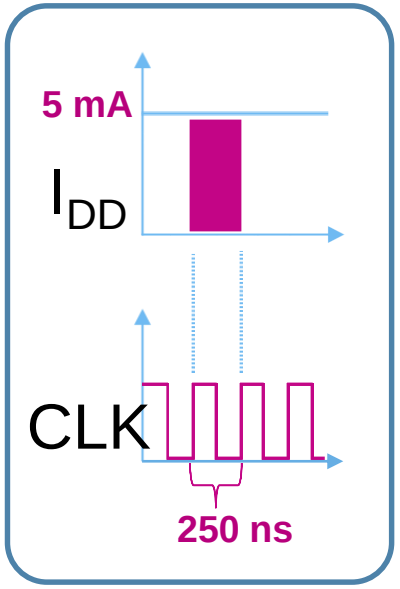


2. From the peripherals

DMA

3. From the compiler

4. From every clock cycle



LOWEST power consumption

HIGHEST performance

Enjoy!



www.st.com/stm32l4